



同濟大學
TONGJI UNIVERSITY

Tongji University

Department of Computer Science and Technology

编译原理目标代码生成实验报告

Author:

An Assignment submitted for the Tongji University:

CS100395 编译原理

2024 年 5 月 20 日

目录

1	需求分析	3
1.1	实验目的	3
1.2	程序任务输入及其要求	3
1.2.1	程序任务输入	3
1.2.2	输入要求	3
1.3	输出形式	4
1.3.1	文法分析输出文件	4
1.3.2	词法分析输出文件	4
1.3.3	语法分析输出文件	4
1.3.4	中间代码生成输出文件	4
1.3.5	目标代码生成输出文件	5
1.4	程序功能	5
2	概要设计	6
2.1	任务分解	6
2.2	数据结构	7
2.2.1	符号表	7
2.2.2	代码生成器	8
2.2.3	词法符号 Token	10
2.2.4	基本块类 Block	11
2.2.5	四元式	12
2.2.6	目标代码数组	13
2.3	主程序流程	13
2.4	模块间的调用关系	14
3	详细设计	15
3.1	类 C 编译器的任务	15
3.2	语义分析过程	16
3.2.1	parser 函数	19
3.3	目标代码生成过程	23
3.3.1	划分基本块算法	23
3.3.2	待用活跃信息表	26

3.3.3	目标代码生成的总控程序	33
3.3.4	目标代码生成函数	35
3.3.5	寄存器分配算法	41
4	调试分析	42
4.1	正确结果演示	42
4.1.1	输入文法	42
4.1.2	源程序文件	42
4.1.3	生成语法树	43
4.2	中间代码生成	44
4.2.1	目标代码生成	44
4.3	错误处理展示	46
5	调试分析	48
5.1	时间复杂度分析	48
5.1.1	语义分析 parser 函数	48
5.1.2	基本块划分	48
5.1.3	目标代码生成	48
5.2	遇到的问题及解决办法	49
5.2.1	函数声明和调用问题	49
5.2.2	符号表内容重复冗余问题	50
5.2.3	存在部分符号无法识别问题	51
5.2.4	出基活跃变量计算错误	51
5.2.5	函数过程调用栈帧难点	54
6	用户使用说明	54
7	总结与收获	56
7.1	收获	56
7.2	遇到的问题与解决思路	57
7.3	在实验过程中对课程的认识	58

1 需求分析

1.1 实验目的

本实验旨在让学生掌握编译器设计与实现的基础技术，通过实践培养以下几方面的能力：

1. 学习如何使用高级程序设计语言实现一个一边扫描的简单语言编译器。
2. 掌握如何实现基本的词法分析器、语法分析器、符号表管理系统、中间代码生成以及目标代码生成。
3. 了解如何将生成的代码正确写入文件中，并保证文件格式的规范性和正确性。

1.2 程序任务输入及其要求

1.2.1 程序任务输入

本程序需要处理两个输入文件：

1. `grammar.txt`：包含类 C 语言的文法定义，该文件详细描述了所支持的类 C 语言的语法规则。
2. `code_in.txt`：包含符合类 C 语言语法的源程序文件，该文件将作为编译器进行分析和处理的输入代码。

1.2.2 输入要求

文法定义文件（`grammar.txt`）使用产生式的形式定义类 C 语言的语法，从根符号 S 开始，通过一系列产生式规则不断扩展。文件中应包含变量定义、函数定义、表达式计算、条件判断、循环结构和函数调用等语法规则。

源程序文件（`code_in.txt`）应严格符合上述文法定义文件中的语法规则，以便编译器能够顺利进行词法分析、语法分析、语义分析，并生成中间代码和目标代码。

1.3 输出形式

编译器的运行过程将生成多个输出文件，用于记录和展示各个阶段的分析和处理结果。

1.3.1 文法分析输出文件

1. Grammar_Rules.txt: 存储编译器对类 C 语言文法产生式的处理结果。
2. Word_Lable.txt: 记录过程中使用到的符号及其对应编号。

1.3.2 词法分析输出文件

1. Pre-Processed_Code.txt: 存储经过预处理的源程序文件，该文件中去除了程序注释和语句之间的多余空格。
2. Word_Lable.txt: 记录词法分析生成的单词种别表，存储格式为：[单词种类]——[种别编码]——[单词符号]。
3. Lexical_Result.txt: 存储单词分割判定后的结果，存储格式为：[单词种类]——[种别编码]——[单词符号]。
4. Lex_to_Parse.txt: 记录词法分析程序送入语法分析的数据。

1.3.3 语法分析输出文件

1. Tables.csv: 记录语法分析生成的 ACTION 表和 GOTO 表。
2. Zero_Closure.txt: 记录语法分析过程中生成的初始闭包。
3. Analysis_Process.txt: 记录对源程序进行语法分析的规约过程，输出格式包括规约步骤序号、状态栈和符号栈。
4. Parse_Tree.png: 语法分析树的图片，以可视化方式展示语法分析过程。

1.3.4 中间代码生成输出文件

1. Quaternion_List.txt: 以四元式形式表达的中间代码序列。

1.3.5 目标代码生成输出文件

1. `Target_Code.asm`: 记录编译器生成的目标代码, 以 MIPS 汇编代码格式展示, 可直接使用 Mars 运行。

1.4 程序功能

本编译器采用一次性编译的方式, 对输入的符合类 C 语言语法的源程序进行完整的编译过程, 主要功能包括:

1. **文法分析**: 读取并分析文法定义文件, 生成相应的文法规则和自动机。
2. **词法分析**: 对源程序进行单词分割和识别, 生成单词种别表和词法分析结果。
3. **语法分析**: 使用自动生成的 ACTION 表和 GOTO 表, 对源程序进行语法分析, 判断其是否符合文法规则, 并记录分析过程。
4. **语义分析**: 在语法分析过程中进行语义检查, 生成四元式形式的中间代码, 检测并报告各种语义错误。
5. **中间代码生成**: 生成四元式表示的中间代码, 进行基本块划分和分析, 附加活跃和待用信息。
6. **目标代码生成**: 将中间代码翻译为 MIPS 汇编代码, 处理寄存器分配和栈帧管理, 生成可执行的目标代码。

编译器首先读取源程序代码和文法定义文件, 生成自动机并进行词法分析, 将源代码分割成单词并进行识别。利用生成的 ACTION 表和 GOTO 表, 对词法分析后的源程序进行语法分析, 判断代码是否符合文法规则, 并输出规约过程。

在语法分析的同时进行语义分析, 生成四元式形式的中间代码。编译器在此过程中可以检测参数未定义、参数重定义、main 函数未定义、函数无返回值、函数调用参数错误等问题, 并报告错误类型和具体信息, 帮助开发者调试代码。

在生成目标代码时, 编译器首先对四元式列表进行基本块划分, 使用函数名进行初步划分, 然后根据条件跳转指令和无条件跳转指令进一步划分基本块。分析基本块的入口和出口, 附加活跃和待用信息。最后, 将四元式翻译为 MIPS 汇编代码, 处理寄存器分配和栈帧管理, 生成能够正确执行的 MIPS 汇编代码。

2 概要设计

2.1 任务分解

语义分析器和中间代码生成主要分为符号表的建立，以及中间代码生成两部分。综合先前的语法分析器来看，在词法分析部分，程序首先读入源程序文件，然后按照预定义的词法规则对源程序进行整理。在语法分析部分，程序首先读入文法产生式，然后利用这些产生式构造对应表项，进行语法分析。

接下来，就是符号表的建立。符号表是一个重要的数据结构，用于存储源程序中出现的各种标识符（如变量名、函数名等）以及它们的相关信息。为了实现符号表的建立，我们采用一个单独的类来表示符号表，每个过程（函数）都会分配一个符号表，用于存放该函数所使用的局部变量和立即数等信息。

中间代码生成需要配合语义分析过程，我们将使用语法制导翻译方法进行处理，对所有的产生式都配备一个翻译程序。这意味着对于每个语法结构，我们通过这些翻译程序执行相关的动作，其中包括生成中间代码和执行必要的语义分析。例如，针对表达式文法的产生式，我们在翻译程序中执行相应的动作，如生成加法和乘法的中间代码，以及对标识符和数字进行处理。通过这种有机的结合，语法制导翻译方法在中间代码生成过程中确保了对源程序语义的准确捕捉，为后续编译流程的正确性奠定了基础。

得到了中间代码生成以后，我们再分三步走。首先是基本块的划分，我们先根据函数进行分块，因为函数之间肯定不再一个基本块中间，因为函数肯定会有跳转语句，`return`，`call` 等等。函数块中，我们遵循以下原则进行基本块的划分：

1. 程序第一个语句
2. 能由条件转移语句或无条件转移语句转移到的语句
3. 紧跟在条件转移语句后面的语句

然后，再划分完基本块之后，我们在每一个基本块中进行待用活跃信息表的建立。待用活跃信息表是针对的每一个基本块中的变量，在开始时，把每个基本块中各变量的符号表等级为非待用，然后根据该变量在基本块出口之后是不是活跃的，把其中的活跃信息栏填写为”活跃“或”非活跃“。在基本块出口到基本块入喉，我们再有后往前一次处理各个四元式。

在建立完待用活跃信息表之后，我们对分块后的中间代码进行翻译，对每一

种比如说 call, return, get, par, +, 语句进行翻译, 并进行寄存器的分配。我将在具体章节中详细说明。

2.2 数据结构

2.2.1 符号表

符号表中包括了符号表 (SymbolTable) 以及表项 (TableItem) 两个结构体, TableItem 结构体包含了每个符号的值 value、类型 type、is_inst、newtype, newname 信息。SymbolTable 结构体中有一个存储表项的索引的 table_entrys, 为每个函数定义的一张自己的符号表 table_list, 用于快速查找符号的索引 table_map_list, 一个用于快速查找函数的索引 funcname2index。

```
1 struct TableItem
2 {
3     std::string value;
4     ArgType type;
5     bool is_inst;
6     //new
7     std::string newtype;
8     std::string newname;
9 };
10
11 class SymbolTable
12 {
13 public:
14
15     std::vector<int> table_entrys;
16     std::vector<std::vector<TableItem>> table_list;
17     std::vector<std::map<std::string, int>> table_map_list;
18     std::map<std::string, int> funcname2index;
19
20     void mkTable(int);
21     int addEntry(std::string, ArgType, bool, std::string, std::string, bool);
22     int addVar(std::string, ArgType, bool, std::string = "", std::string = "");
23     int addGlobalVar(std::string, ArgType, bool, std::string = "", std::string = "");
24     std::string getName(int);
25     void displayTable(std::ofstream&);
26
27     void clear();
28
29 private:
30
31 };
```


2.2.2 代码生成器

Code 结构体包括操作符 op、操作数 arg1、arg2、result，以及标记是否为全局变量的 arg1_p、arg2_p、result_p。此外，新增了用于存储字符串表示的操作数 sarg1、sarg2、sresult，以及记录上一次操作的索引 last_index。

```
1 struct Code
2 {
3     //Tag op;
4     std::string op;
5     int arg1;
6     int arg2;
7     int result;
8     int arg1_p;    // 是否为全局变量
9     int arg2_p;
10    int result_p;
11};
```

CodeGenerator 类包含错误 Token 对象 err_token、存储中间代码的 code_list 和 tarCode_list。它提供了私有方法和辅助方法，用于生成中间代码、处理变量的存储位置和类型等操作。该类还包括一系列用于语法翻译的方法，如声明变量、获取变量位置类型、生成赋值语句等。它定义了一系列处理不同语法结构的私有方法，如声明函数、程序、保留字操作等。此外，还提供了一系列处理逻辑和控制流的方法，如逻辑运算、条件语句、循环语句、返回语句等。类内维护了一个 translate_map，将产生式映射到对应的处理方法。最后，提供了写入中间代码到文件和输出中间代码到流的方法。

```
1 class CodeGenerator
2 {
3 private:
4     Token err_token;
5     std::vector<Code> code_list;
6     std::vector<TarCode> tarCode_list;
7 public:
8     State transDeclareVar(Token& left, const vector<Token>& right);
9 }
```

```

10 private:
11     string getVariableDescription(const vector<Token>& right) const;
12     string getMemoryDescription(int memorySize) const;
13     void updateFunctionParameterCount();
14     void initializeFunctionEnvironment();
15     bool getVariablePlace(const Token& token, int& place, int& resultPlace, string& resultStr)
16         ;
17     string getVariableName(const Token& token);
18     void genAssignStatement(int arg1Place, int place, int resultPlace, const string& arg1Str,
19         const string& resultStr);
20     int getVariablePlaceType(const Token& token);
21
22 private:
23     //一系列辅助方法
24     void gen(std::string, int, int, int, int = CodeGenerator::ARG_LOCALVAR, int =
25         CodeGenerator::ARG_LOCALVAR, int = CodeGenerator::ARG_LOCALVAR, string = "", string = "",
26         string = "", int = 0);
27
28     int merge(int list_1, int list_2);
29     void backpatch(int back_list, int quad);
30
31     State transGetpos(Token&, const std::vector<Token>&);
32     State transMktable(Token&, const std::vector<Token>&);
33
34     State transDeclareFunc(Token&, const std::vector<Token>&);
35     State transDeclareProgram(Token&, const std::vector<Token>&);
36
37     State transStatKeep(Token&, const std::vector<Token>&);
38     State transStatStr2StatStr(Token&, const std::vector<Token>&);
39
40     State transAssign(Token&, const std::vector<Token>&);
41     State transAssignKeep(Token&, const std::vector<Token>&);
42     State transAssignBina(Token&, const std::vector<Token>&);
43     State transAssignCmp(Token&, const std::vector<Token>&);
44
45     State transBoolKeep(Token&, const std::vector<Token>&);
46     State transBoolOr(Token&, const std::vector<Token>&);
47     State transBoolAnd(Token&, const std::vector<Token>&);
48     State transBoolNot(Token&, const std::vector<Token>&);
49     State transBoolCmp(Token&, const std::vector<Token>&);
50
51     State transDo_jump(Token&, const std::vector<Token>&);
52     State transIf_then(Token&, const std::vector<Token>&);
53     State transIf_then_else(Token&, const std::vector<Token>&);
54     State transWhile(Token&, const std::vector<Token>&);
55     State transReturn(Token&, const std::vector<Token>&);
56
57     State transFactor_Func(Token&, const std::vector<Token>&);
58     State transPreExpr_ExprPreExpr(Token&, const std::vector<Token>&);
59     State transArgulist_ExprPreExpr(Token&, const std::vector<Token>&);

```

```

57     State transArr(Token&, const std::vector<Token>&);
58     State transParam(Token&, const std::vector<Token>&);
59
60 public:
61     std::map<int, State( CodeGenerator::*)(Token&, const std::vector<Token>&)> translate_map;
62     CodeGenerator();
63     State writeCodeToFile(const char* symbol_table_file, const char* middle_code_file);
64     State writeCode(ostream& des, int l, int r);
65     const Token& getErrorToken();
66     State clear_code();
67 };

```

2.2.3 词法符号 Token

对前一个项目语义分析器定义的 **Token** 数据结构进行优化，在原有词法单元的属性值以及对应的词法单元的类型的基础上，增加了存放属性和错误信息的变量。

```

1 struct Token
2 {
3     std::string value;
4     TokenInfo info;
5     Tag tag;
6     VN_attrs attrs;
7
8 };
9
10 union VN_attrs
11 {
12     Varattrs    var_attrs;
13     Boolattrs   bool_attrs;
14     Ctrlattrs   ctrl_attrs;
15     Funattrs    fun_attrs;
16 };
17
18 struct Varattrs
19 {
20
21     int table_i;
22     int item_i;
23     int place;
24     ArgType type;
25 };
26
27 struct Boolattrs

```

```

28 {
29     int t_list;
30     int f_list;
31 };
32
33 struct Ctrlattrs
34 {
35     int next_list;
36     int quad;
37 };
38
39 struct Funattrs
40 {
41     int param_num;
42 };
43
44
45 struct TokenInfo
46 {
47     int line;
48     int col;
49     std::string why;
50 };

```

2.2.4 基本块类 Block

基本块类主要用于记录基本块的基本信息。这个类包括以下几个关键组成部分：

1. 基本块的名称：每个基本块都有一个唯一的名称，用于标识该基本块。这有助于在程序中引用和操作特定的基本块。

2. 基本块中的四元式序列：基本块内部包含一系列的四元式（四元运算符）。这些四元式表示了程序的具体操作，如赋值、算术运算和条件跳转等。

3. 后继基本块：由于四元式可能包含有条件跳转语句，因此每个基本块需要记录其后继的基本块。后继基本块是指在执行完当前基本块之后程序可能会跳转到的基本块。为了支持这一功能，我们设置了两个参数，分别用于记录基本块的后继基本块。

记录后继基本块的信息非常重要，在后续计算基变量和入基变量的活跃状态时将会用到这些信息。具体来说，基变量和入基变量的活跃性分析依赖于程序的控制流图，而控制流图是通过基本块之间的跳转关系建立起来的。因此，准确记

录和管理基本块的后继关系对于程序优化和分析来说是至关重要的。

```
1 struct IBlock {  
2     string name;  
3     vector<QuaternaryWithInfo> codes;  
4     int next1;  
5     int next2;  
6 };
```

2.2.5 四元式

QuaternaryWithInfo 类是一个用于表示四元式以及相关变量信息的数据结构。这个类包含一个四元式（Quaternary）对象和三个变量信息（VarInformation）对象。以下是对这个数据结构的详细介绍：

成员变量 Quaternary q: 这是一个四元式对象，表示一个四元运算符。四元式通常用于表示中间代码中的基本操作，格式为 (操作符, 操作数 1, 操作数 2, 结果)。例如, (get, _, _, a) 表示一个将值赋给变量 a 的操作。

VarInformation info1: 这是一个与四元式相关的变量信息对象，用于存储与四元式左操作数相关的信息。

VarInformation info2: 这是一个与四元式相关的变量信息对象，用于存储与四元式右操作数相关的信息。

VarInformation info3: 这是一个与四元式相关的变量信息对象，用于存储与四元式结果变量相关的信息。

```
1 class QuaternaryWithInfo { // (get, _, _, a) (^, ^) (^, ^) (l, y) 这个形式 左操, 右操, 左值  
2 public:  
3     Quaternary q;  
4     VarInformation info1;  
5     VarInformation info2;  
6     VarInformation info3;  
7  
8     QuaternaryWithInfo(Quaternary q, VarInformation info1, VarInformation info2, VarInformation  
9         info3);  
10    void output(ostream& out);  
11 };
```

2.2.6 目标代码数组

用于储存目标代码，是一个 `string` 类型的数组，所以在 `push` 的时候，我们需要手动设置格式

```
vector<string>objectCodes;
```

2.3 主程序流程

词法分析器与语法分析器的流程在此就不再赘述，下面介绍中间代码生成器的主要操作流程，在语法分析的过程当中，如果当前扫描到的需要规约，并且当前产生式在 `codeGenerator` 中有相应的动作，那么就调用动作。调用动作的过程有一个返回值，接受这个返回值，来表示当前语义分析的正确性。如果接收到语义分析正确的返回值，那么继续往后扫描，重复上述动作；如果语义分析错误，则报错跳出语法分析过程。

在生成中间代码后，我们按三步流程进行处理。首先，根据函数进行基本块划分，遵循程序第一个语句、转移目标语句、和条件转移语句后续语句的原则。然后，为每个基本块建立待用活跃信息表，初始化变量状态并根据其在基本块出口后的活跃性更新状态。从基本块出口向入口反向处理每个四元式。最后，对分块后的中间代码进行详细翻译，针对不同语句类型分配寄存器，这些步骤将在具体章节中详细说明。

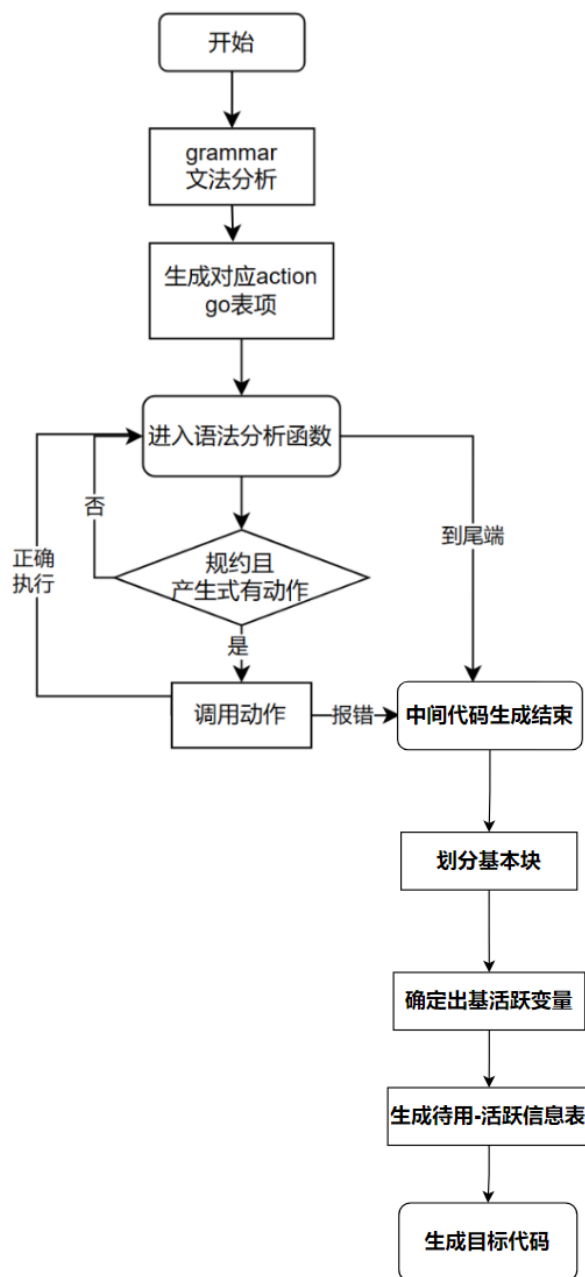


图 1: 主程序流程图

2.4 模块间的调用关系

本次项目的主要模块为词法分析器、中间代码生成器、语法分析器和符号表，基本块，待用-活跃信息表构建，。对比前面的实验项目，主要增加了基本块和目标代码的生成。

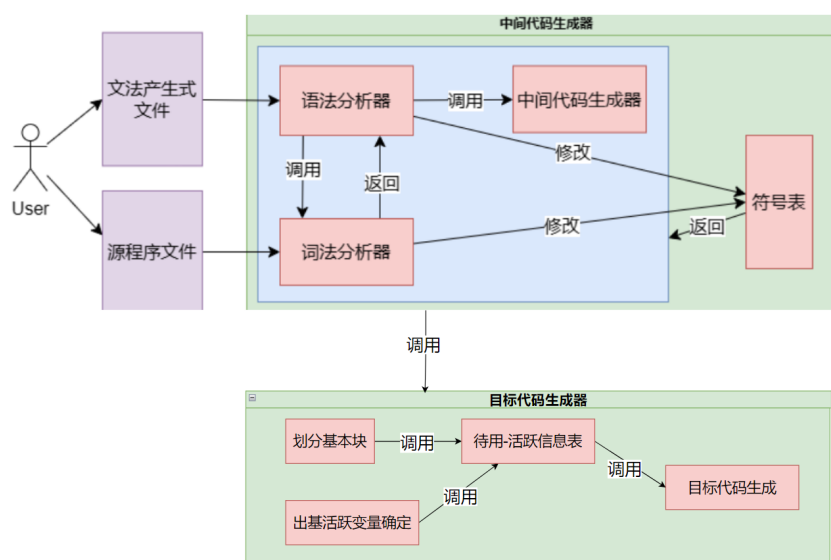


图 2: 模块间调用关系

3 详细设计

3.1 类 C 编译器的任务

语义分析的主要目标是对源程序的语义进行分析和判断。检查代码中是否存在错误，比如未经声明直接使用某变量名、错误的布尔表达式和返回类型等语法错误。

当检测到语法错误，系统将产生相应的错误信息。本次实验项目完成的中间代码生成器，主要实现了将源程序的代码翻译成中间代码，并由语义分析器在翻译的时候对源程序进行一定程度的语义分析的功能。

中间代码生成是构建编译器的关键步骤之一，它在语法分析的基础上为源程序生成中间表示，为后续优化和目标代码生成打下基础。这一过程中，我们采用语法制导翻译方法，通过为每个需要执行特定操作的文法产生式关联相应的语义动作。

在进行语法分析时，当对产生式进行归约时，会调用其翻译函数执行相应的语义动作。这些动作包括操作符的计算、变量和常量的存储等。而为了生成中间代码，我们并不立即将翻译语句写入文件，而是将其暂时存储在一个特定的数据结构中，即四元式结构体。

将中间代码转换为目标代码前，进行基本块的划分和待用活跃信息表的建立是至关重要的步骤。首先，基本块的划分依照以下原则：首先是程序的第一个语

句；其次，任何可以通过条件或无条件转移语句到达的语句；最后，条件转移语句后的直接语句。这种划分确保了每个基本块内部的流控制结构的独立性。

接下来，对于每个基本块，我们需要构建待用活跃信息表。这个表涉及到基本块内的每个变量，一开始，所有变量都标记为“非待用”。随后，根据变量在基本块出口后是否活跃，更新活跃信息为“活跃”或“非活跃”。

最后，基于这些基本块和待用活跃信息，进行中间代码到目标代码的翻译。这包括对各种语句（如 `call`、`return`、`get`、`par`、加法等）的处理和寄存器分配，以确保生成的代码既高效又优化。

下面我将着重介绍语义分析过程和目标代码生成过程。

3.2 语义分析过程

语义分析是编译过程中一项关键任务，其目标在于深入分析源程序的语义，以发现并判断潜在的错误操作。这类错误包括但不限于未经声明直接使用某变量名、存在错误的布尔表达式、以及返回类型不符合规范等。语义分析通过对源程序的全面审查，有助于提前发现并纠正潜在的逻辑和语义错误，提高编译程序的稳健性和可靠性。

在语义分析过程中，语法制导翻译方法扮演着至关重要的角色。具体而言，为了进行语义检查，我们为所有需要的产生式配备了相应的翻译程序。这些翻译程序负责检查源程序中的各种语义问题，包括但不限于类型检查、变量是否正确定义等方面。一旦检测到错误，系统会立即记录错误的位置和相关符号，并迅速返回相应的错误信息，以协助程序员定位和修复问题。

与此同时，对于通过了语义检查的代码部分，语义分析还会执行一系列后续动作。其中包括维护符号表，以确保变量和标识符的正确使用，以及生成中间代码，为后续的编译阶段提供有序而规范的数据。通过这些步骤，语法制导翻译方法不仅有助于减少源程序中的错误，也为后续编译步骤提供了清晰的语义基础，使得整个编译过程更加高效和可靠。

该函数用于生成中间代码四元式，并将其添加到代码列表 `code_list` 中。函数参数包括操作符、操作数 1、操作数 2、结果，以及它们是否为全局变量 `arg1_p`、`arg2_p`、`result_p`，以及它们的字符串表示形式和上一个四元式的索引。该函数通过将生成的四元式添加到 `code_list` 中，记录了程序执行的中间状态。

```

1 void CodeGenerator::gen(string op, int arg1, int arg2, int result, int arg1_p, int arg2_p, int
   result_p, string sarg1, string sarg2, string sresult, int lasti)
2 {
3     code_list.push_back({ op, arg1, arg2, result, arg1_p, arg2_p, result_p, sarg1, sarg2,
   sresult, lasti });
4 }

```

该函数用于将两个中间代码序列合并。参数 `list_1` 和 `list_2` 分别表示两个中间代码序列的头部。如果 `list_2` 不为默认位置 (`DEFAULT_LOC`)，则将 `list_1` 接在 `list_2` 的末尾。返回新的合并后的序列的头部。

```

1 int CodeGenerator::merge(int list_1, int list_2)
2 {
3     if (list_2 != DEFAULT_LOC)
4     {
5         int p = list_2;
6         while (code_list[p].result != DEFAULT_LOC)
7             p = code_list[p].result;
8         code_list[p].result = list_1;
9         return list_2;
10    }
11    else
12        return list_1;
13 }

```

该函数用于回填中间代码序列中的跳转目标。参数 `back_list` 表示需要回填的序列的头部，`quad` 表示跳转目标的四元式索引。该函数遍历 `back_list` 中的每个四元式，将它们的结果字段回填为 `quad`。

```

1 void CodeGenerator::backpatch(int back_list, int quad)
2 {
3     if (back_list == DEFAULT_LOC)
4         return;
5     int p = back_list;
6     while (code_list[p].result != DEFAULT_LOC)
7     {
8         int p_tmp = code_list[p].result;
9         code_list[p].result = quad;
10        p = p_tmp;
11    }
12    code_list[p].result = quad;

```

```
13 }
```

下面是编译器中用于处理函数声明语句的转换函数，负责处理两种类型的函数声明语句：

- `int id ( params ) in_dec stat_str`
- `void id ( params ) in_dec stat_str`

```
1 State CodeGenerator::transDeclareFunc(Token& left, const vector<Token>& right) {
2     // dec -> int id ( mhtable params ) { in_dec stat_str }
3     // dec -> void id ( mhtable params ) { in_dec stat_str }
4
5     const string& funcName = right[1].value;
6
7     if (symbol_table.funcname2index.count(funcName)) {
8         err_token = right[1];
9         return State::ERROR; // 不允许重载
10    }
11
12    size_t tableIndex = symbol_table.table_list.size() - 1;
13    symbol_table.funcname2index[funcName] = tableIndex;
14
15    const ArgType funcType = (STR2TAG.at(right[0].value) == Tag::kw_int) ? ArgType::t_int_func
16        : ArgType::t_void_func;
17    symbol_table.addGlobalVar(funcName, funcType, false, TAG2STR.at(right[0].tag) + " fun-" +
18        to_string(right[4].attrs.fun_attrs.param_num), "F" + to_string(tableIndex));
19
20    auto& functionEntry = symbol_table.table_list.back()[0];
21    functionEntry.value = funcName;
22    functionEntry.type = funcType;
23
24    // 回填函数名
25    auto& codeEntry = code_list[symbol_table.table_entries.back()];
26    codeEntry.op = "F" + to_string(tableIndex) + ":";
27    codeEntry.arg1 = DEFAULT_LOC;
28    codeEntry.arg2 = DEFAULT_LOC;
29    codeEntry.result = DEFAULT_LOC;
30
31    if (code_list.back().op != "return") {
32        gen("return", DEFAULT_LOC, DEFAULT_LOC, DEFAULT_LOC);
33    }
34    return State::OK;
35 }
```

具体步骤如下：

1. 提取函数名 `funcName`。
2. 检查函数名是否已经存在于函数表中，如果存在则返回错误，因为不允许函数重载。
3. 获取当前符号表的索引作为函数表的索引，并将函数名与索引映射关系记录在 `funcname2index` 中。
4. 根据函数的返回类型（`int` 或 `void`）确定函数的类型为 `ArgType::t_int_func` 或 `ArgType::t_void_func`。
5. 在符号表中添加全局变量，表示函数的声明，其中变量名为函数名，类型为函数类型，不是数组，描述为函数参数信息，生成的中间代码标签为 `F` 加上函数表的索引。
6. 更新符号表中的函数表的第一个表项，将其值设置为函数名和类型。
7. 回填函数名到中间代码中，即将代码列表中函数声明的地址 `F + 表索引` 更新为实际的函数名。
8. 检查是否需要在函数结尾添加返回语句，如果最后一条中间代码不是返回语句，则添加一个默认的返回语句。

该函数的目的是在符号表和中间代码中正确处理函数声明，包括函数名的唯一性、符号表的维护和中间代码的生成。

3.2.1 parser 函数

对比第一个实验项目的语法分析器，在本次中间代码生成器项目的编写过程中，在语法分析函数部分增加了对于中间代码生成器类的调用，生成对应的中间代码。

首先在语法分析函数中，将数字或是标识符的文法符号加入到符号表中：

```
1 if (temp_token.tag == Tag::id)
2 {
3     temp_token.attrs.var_attrs.place = -1;
4 }
5 else if (temp_token.tag == Tag::num)
6 {
```

```

7   if (symbol_table.table_map_list.back().count(temp_token.value))
8       temp_token.attrs.var_attrs.place = symbol_table.table_map_list.back()[temp_token.value];
9   else
10      temp_token.attrs.var_attrs.place = symbol_table.addVar(temp_token.value, ArgType::t_int_var, true, temp_token.value, "int inst");
11      temp_token.attrs.var_attrs.item_i = temp_token.attrs.var_attrs.place;
12      temp_token.attrs.var_attrs.table_i = symbol_table.table_list.size() - 1;
13      temp_token.attrs.var_attrs.type = ArgType::t_int_var;
14  }

```

在语法分析的归约操作中调用相应的代码生成函数，将产生式翻译成中间代码，具体步骤如下：

1、code_generator.translate_map.count(temp_move.go)：检查 temp_move.go 是否在 code_generator.translate_map 中，即检查是否存在与当前产生式编号对应的翻译函数。temp_move.go 是产生式的编号，用于确定应该执行哪个翻译函数。

2、code_generator.translate_map[temp_move.go]：如果产生式编号存在于映射中，这将返回与该编号相关联的翻译函数的指针。

3、(code_generator.*code_generator.translate_map[temp_move.go])(left_now, right_list)：通过指针调用翻译函数。其中 code_generator.translate_map[temp_move.go] 返回的是成员函数指针，code_generator.* 用于解引用该指针，然后调用函数。

4、(code_generator.*code_generator.translate_map[temp_move.go])(left_now, right_list) == State::ERROR：检查翻译函数的执行结果是否为错误状态。如果翻译函数返回 State::ERROR，说明翻译过程中发生了错误。

```

1  if (code_generator.translate_map.count(temp_move.go))
2      if ((code_generator.*code_generator.translate_map[temp_move.go])(left_now, right_list) == State::ERROR) {
3          err_token = code_generator.getErrorToken();
4          return -1;
5      }

```

parser 函数完整代码：

```

1  int parser(Lexer& lexer, const char* src_path, Token& err_token)
2  {
3
4      stack<int> state_stack; // 状态栈
5      stack<Token> in_stack; // 输入栈

```

```

6   stack<int> node_Stack; //树结点栈，存放树节点下标
7
8   state_Stack.push(0); //初始化
9   in_Stack.push({ Tag::the_end }); //初始化
10  //NStack.push(-1); //初始化
11
12  bool islast_Token = false; //判断是否使用上次的token
13  Token temp_token; //当前token
14  int temp_state; //当前state
15  Movement temp_move; //当前动作
16  while (true) {
17
18      //需要新获取一个token
19      if (!islast_Token) {
20          if (lexer.getNextLexical(temp_token) == -1)
21          {
22              return -1;
23          }
24          /*-----new
-----*/
25          //将num和标识符插入到符号表中
26          if (temp_token.tag == Tag::id)
27          {
28              temp_token.attrs.var_attrs.place = -1;
29          }
30          else if (temp_token.tag == Tag::num)
31          {
32              if (symbol_table.table_map_list.back().count(temp_token.value))
33                  temp_token.attrs.var_attrs.place = symbol_table.table_map_list.back()[
temp_token.value];
34              else
35                  temp_token.attrs.var_attrs.place = symbol_table.addVar(temp_token.value ,
ArgType::t_int_var , true , temp_token.value , "int inst");
36              temp_token.attrs.var_attrs.item_i = temp_token.attrs.var_attrs.place;
37              temp_token.attrs.var_attrs.table_i = symbol_table.table_list.size() - 1;
38              temp_token.attrs.var_attrs.type = ArgType::t_int_var;
39          }
40      }
41      /*-----end-----*/
42      temp_state = state_Stack.top();
43      cout << temp_token.value << "*" << findTagName(temp_token.tag) << endl;
44      if (action_go_map.count(temp_state) == 0 || action_go_map[temp_state].count(temp_token
.tag) == 0) {
45          //若对应表格项为空,则出错
46          cout << action_go_map[temp_state].count(temp_token.tag) << "*" << endl;
47          cout << action_go_map.count(temp_state) << "*" << endl;
48          err_token = temp_token;
49
50          return -1;
51      }
52      temp_move = action_go_map[temp_state][temp_token.tag];

```

```

53 // 归约操作
54 if (temp_move.action == Action::reduction) {
55     int len = grammars[temp_move.go].right.size();
56
57     TNode node_left;
58     node_left.ntag = grammars[temp_move.go].left;
59     node_left.p = pTree.TNode_List.size();
60
61
62     vector<Token> right_list(len);
63     // 移出栈
64     while (len-- > 0) {
65         state_stack.pop();
66
67         right_list[len] = in_stack.top();
68         in_stack.pop();
69
70         node_left.childs.push_front(node_stack.top()); // 创建子结点链表
71         node_stack.pop();
72     }
73
74     Token left_now = { node_left.ntag };
75     // 调用对应函数
76     if (code_generator.translate_map.count(temp_move.go))
77         if ((code_generator.*code_generator.translate_map[temp_move.go])(left_now,
right_list) == State::ERROR) {
78             err_token = code_generator.getErrorToken();
79             return -1;
80         }
81
82
83     pTree.TNode_List.push_back(node_left); // 移进树栈
84
85     temp_state = state_stack.top(); // 更新当前状态
86     if (action_go_map.count(temp_state) == 0 ||
87         action_go_map[temp_state].count(node_left.ntag) == 0) {
88         // 若对应表格项为空, 则出错
89         err_token = temp_token;
90         cout << 5 << endl;
91         return -1;
92     }
93
94     temp_move = action_go_map[temp_state][node_left.ntag]; // 更新当前动作
95     // 入栈操作
96     islast_token = true;
97     state_stack.push(temp_move.go);
98     in_stack.push(left_now);
99     node_stack.push(node_left.p);
100
101
102 }

```

```

103     // 移进操作
104     else if (temp_move.action == Action::shift_in) {
105         state_stack.push(temp_move.go);
106         in_stack.push(temp_token);
107
108         TNode in_node;
109         in_node.ntag = temp_token.tag;
110         in_node.p = pTree.TNode_List.size();
111         pTree.TNode_List.push_back(in_node); // 移进树结点
112         node_stack.push(in_node.p);
113
114         islast_token = false;
115     }
116     else // 接受
117     {
118         pTree.RootNode = pTree.TNode_List.size() - 1;
119         return 1; // accept
120     }
121 }
122 }

```

3.3 目标代码生成过程

3.3.1 划分基本块算法

首先，算法从函数的入口点开始，这些入口点标记了每个函数的开始。为每个函数创建一个基本块的列表，并且初始化一个优先队列来记录基本块的入口点，这些入口点是基于控制流的转移点，如跳转和返回指令。然后，对于每个函数，算法遍历其代码指令，特别关注跳转（如无条件跳转和条件跳转）、调用和返回指令。每当遇到这些控制流指令时，就会将目标代码行作为新的基本块入口点添加到队列中。对于条件跳转指令，除了跳转目标外，紧跟其后的指令也被视为新的基本块入口点。

遍历完成后，使用优先队列中的入口点，算法开始构建基本块。从函数的第一个入口点开始，将连续的指令集合到当前基本块中，直到下一个入口点。每个新的入口点标记了一个基本块的结束和下一个基本块的开始。如果一个基本块是由跳转指令结束的，它会记录下一个可能的基本块（即跳转的目标），以及顺序执行的下一个基本块。

对于每个基本块，根据它是否是函数的第一个基本块来赋予它一个名字。如

果是第一个基本块，则使用函数名；否则，使用生成的新标签。这些基本块随后被存储在与函数名对应的数据结构中，为后续的优化和代码生成步骤做准备。此外，每个基本块的末尾指令（尤其是控制流指令）决定了它可能的后继基本块，从而构建了整个函数的控制流图。

```

1 void IntermediateCode::divideBlocks(vector<pair<int, string>> funcEnter) {
2
3     cout << " * *****";
4     for (const auto& entry : funcEnter) {
5
6         std::cout << "Function start at line: " << entry.first << ", Name: " << entry.second <<
          std::endl;
7     }
8
9     // 遍历 vector 并打印每个四元组指令的内容
10    for (const auto& instr : code) {
11        std::cout << "Operation: " << instr.op << ", "
12            << "Source1: " << instr.src1 << ", "
13            << "Source2: " << instr.src2 << ", "
14            << "Destination: " << instr.des << std::endl;
15    }
16
17    cout << " * *****";
18    //code就是中间代码
19    //1.函数的第一个语句
20    //2.能由条件转移语句或无条件转移语句转移到的语句
21    //3.紧跟在条件转移语句后面的语句
22
23    //先划分函数块
24    //再划分基本块 基本块带有两个参数，指明它下面一个基本块是什么，如果是跳转指令，将指明之后的
      在它时候的两个基本块分别是什么
25
26    for (vector<pair<int, string>>::iterator iter = funcEnter.begin(); iter != funcEnter.end();
          iter++) { // 对每一个函数块
27        vector<Block> blocks;
28        priority_queue<int, vector<int>, greater<int>> > block_enter; // 记录所有基本块的入口点
29        block_enter.push(iter->first);
30
31        int endIndex = iter + 1 == funcEnter.end()? code.size(): (iter + 1)->first;
32        for (int i = iter->first; i != endIndex; i++) {
33            if (code[i].op[0] == 'j') {
34                if (code[i].op == "j") { // 若操作符是j 无条件转移
35                    block_enter.push(atoi(code[i].des.c_str())); // 将destination字段加入
36                }
37            } else { // 若果操作符是j=, j!=, j>=, j>, j<=, j<
38                if (i + 1 < endIndex) {
39                    block_enter.push(i + 1);
40                }
41                block_enter.push(atoi(code[i].des.c_str()));
42            }

```

```

43     }
44     else if (code[i].op == "return" || code[i].op == "call") {
45         if (i + 1 < endIndex) {
46             block_enter.push(i + 1);
47         }
48     }
49 }
50
51 //划分基本块
52 Block block;
53 map<int, string>labelEnter; //入口点和标签的对应关系
54 map<int, int>enter_block; //建立入口点和block的对应关系
55 int firstFlag = true; //函数块第一块标记, 该块命名为函数名
56 int enter;
57 int lastEnter = block_enter.top();
58 block_enter.pop();
59 while (!block_enter.empty()) {
60     //插入四元式到block中
61     enter = block_enter.top();
62     block_enter.pop();
63
64     if (enter == lastEnter) {
65         continue;
66     }
67
68     for (int i = lastEnter; i != enter; i++) {
69         block.codes.push_back(code[i]);
70     }
71
72     if (!firstFlag) { //该基本块不是函数块的第一块基本块
73         block.name = nl.newLabel();
74         labelEnter[lastEnter] = block.name;
75     }
76     else { //该基本块是函数块的第一块基本块
77         block.name = iter->second;
78         firstFlag = false;
79     }
80     enter_block[lastEnter] = blocks.size();
81     blocks.push_back(block);
82     lastEnter = enter;
83     block.codes.clear();
84 }
85 if (!firstFlag) { //该基本块不是函数块的第一块基本块
86     block.name = nl.newLabel();
87     labelEnter[lastEnter] = block.name;
88 }
89 else { //该基本块是函数块的第一块基本块
90     block.name = iter->second;
91     firstFlag = false;
92 }
93 if (iter + 1 != funcEnter.end()) { //在两个函数的起点之间

```

```

94     for (int i = lastEnter; i != (iter+1)->first; i++) {
95         block.codes.push_back(code[i]);
96     }
97 }
98 else { //在最后一个函数至中间代码末尾
99     for (int i = lastEnter; i != code.size(); i++) {
100         block.codes.push_back(code[i]);
101     }
102 }
103 enter_block[lastEnter] = blocks.size();
104 blocks.push_back(block);
105
106 int blockIndex = 0;
107 for (vector<Block>::iterator bIter = blocks.begin(); bIter != blocks.end(); bIter++,
    blockIndex++) {
108     vector<Quaternary>::reverse_iterator lastCode = bIter->codes.rbegin();
109     if (lastCode->op[0] == 'j') {
110         if (lastCode->op == "j") { //若操作符是j
111             bIter->next1 = enter_block[atoi(lastCode->des.c_str())];
112             bIter->next2 = -1;
113         }
114         else { //若果操作符是j=-, j!=, j>=, j>, j<=, j<
115             bIter->next1 = blockIndex + 1;
116             bIter->next2 = enter_block[atoi(lastCode->des.c_str())];
117             bIter->next2 = bIter->next1 == bIter->next2 ? -1 : bIter->next2;
118         }
119         lastCode->des = labelEnter[atoi(lastCode->des.c_str())];
120     }
121     else if (lastCode->op == "return") {
122         bIter->next1 = bIter->next2 = -1;
123     }
124     else {
125         bIter->next1 = blockIndex + 1;
126         bIter->next2 = -1;
127     }
128 }
129 }
130 }
131
132 funcBlocks[iter->second] = blocks;
133 }
134 }

```

3.3.2 待用活跃信息表

首先，算法初始化用于收集每个基本块中定义（DEF）和使用（USE）变量的集合。接着，对每个基本块内的四元式指令进行迭代，标识出每个操作数是被定


```

21 //函数块，每个函数块中包含多个基本块
22 for (map<string , vector<Block> >::iterator fbiter = funcBlocks->begin(); fbiter !=
    funcBlocks->end(); fbiter++) {
23     vector<IBlock> iBlocks;
24     vector<Block>& blocks = fbiter->second;
25
26     // 遍历 vector 并打印每个四元组指令的内容
27     //我们下面的处理对象是blocks，一个blocks对应一个基本块
28     cout << fbiter->first << endl;
29     for (const Block& block : blocks) {
30         printBlock(block);
31         cout << "-----" << std::endl; // 添加分隔符以区分不同的块
32     }
33
34     vector<set<string> >INL, OUTL, DEF, USE;
35
36     //活跃变量的数据流方程
37     //确定DEF定义，USE使用
38     for (vector<Block>::iterator biter = blocks.begin(); biter != blocks.end(); biter++) {
39         set<string>def, use;
40         for (vector<Quaternary>::iterator citer = biter->codes.begin(); citer != biter->codes.
            end(); citer++) {
41             if (citer->op == "j" || citer->op == "call") {
42                 //pass
43             }
44             else if (citer->op[0] == 'j') { //j>= j<=,j==,j!=,j>,j< (j>,i,100,Label5)
45                 if (isVar(citer->src1) && def.count(citer->src1) == 0) { //如果源操作数1还没有被定值
46                     use.insert(citer->src1);
47                 }
48                 if (isVar(citer->src2) && def.count(citer->src2) == 0) { //如果源操作数2还没有被定值
49                     use.insert(citer->src2);
50                 }
51             }
52             else { //(+,a,2,T5)
53                 if (isVar(citer->src1) && def.count(citer->src1) == 0) { //如果源操作数1还没有被定值
54                     use.insert(citer->src1);
55                 }
56                 if (isVar(citer->src2) && def.count(citer->src2) == 0) { //如果源操作数2还没有被定值
57                     use.insert(citer->src2); //未被定值，就引用
58                 }
59                 if (isVar(citer->des) && use.count(citer->des) == 0) { //如果目的操作数还没有被引用
60                     def.insert(citer->des); //未被引用，就定值（新变量）
61                 }
62                 /*cout << "def:" << std::endl;
63                 printSet(def);
64                 cout << "use:" << std::endl;
65                 printSet(use);
66
67                 cout << "-----" << endl;*/
68             }
69         }
    }

```

```

70
71     INL.push_back(use);
72     DEF.push_back(def);
73     USE.push_back(use);
74     OUTL.push_back(set<string>());
75
76 }
77
78
79 //cout << "\nOUTL加了个空:" << std::endl;
80 //printVectorOfSets(OUTL);
81 ///cout << "-----" << endl;
82
83 // 确定INL, OUTL
84 // 目的是计算每个基本块的活跃变量的入集 (INL) 和出集 (OUTL)
85 // 活跃变量是指在某点之后还会被使用到的变量。
86
87 /* 活跃变量分析的基本算法解释:
88     初始化和循环条件:
89
90     change 是一个标志, 用来指示是否有变量的IN集或OUT集在迭代中有变化, 如果有变化, 则需要继续
91     迭代直到没有更多变化, 这意味着达到了一个固定点。
92     while (change) 循环会持续执行, 直到一个完整的遍历中没有发现任何IN集或OUT集的变化。
93     迭代基本块:
94
95     循环遍历每个基本块, 并对每个基本块的IN集和OUT集进行更新。blockIndex 用作当前基本块的索引。
96     更新OUT集和IN集:
97
98     对于每个基本块, 根据其控制流后继 (next1 和 next2) 更新OUT集:
99     . 如果next1不是 - 1 (表示存在有效的后继基本块), 则遍历该后继基本块的IN集, 将所有变量
100     添加到当前基本块的OUT集中。
101     . 类似地处理next2。
102     在将变量从后继基本块的IN集添加到当前基本块的OUT集后, 检查这些变量是否不在当前基本块的
103     DEF集中:
104     . 如果一个变量不在DEF集中, 这意味着它在基本块中没有被重新定义, 因此它需要从当前基本块
105     的IN集中可访问, 以确保它在使用前是活跃的。
106
107     检查和标记变化:
108     . 每当OUT集有新元素被添加时, 检查是否需要更新IN集。
109     . 如果IN集被更新 (即添加了新元素), 将change标志设为true, 表示需要进行另一轮迭代。*/
110
111 //IN集合, 入基后活跃的变量 在基本块开始前, 必须已经赋值且在基本块内部或之后某处使用的变
112 量集合。 所以IN的初始值是use, 就是在该基本块中使用的
113 //OUTL集合, 出基后活跃的变量
114 bool change = true;
while (change) {
    change = false;
    int blockIndex = 0;
    for (vector<Block>::iterator biter = blocks.begin(); biter != blocks.end(); biter++,

```

```

blockIndex++) {
115     int next1 = biter->next1;
116     int next2 = biter->next2;
117     if (next1 != -1) { //有后续块
118         for (set<string>::iterator inlIter = INL[next1].begin(); inlIter != INL[next1].end()
; inlIter++) {
119             if (OUTL[blockIndex].insert(*inlIter).second == true) {
120                 if (DEF[blockIndex].count(*inlIter) == 0) {
121                     INL[blockIndex].insert(*inlIter);
122                 }
123                 change = true;
124             }
125         }
126     }
127     if (next2 != -1) { //有后续块
128         for (set<string>::iterator inlIter = INL[next2].begin(); inlIter != INL[next2].end()
; inlIter++) {
129             if (OUTL[blockIndex].insert(*inlIter).second == true) {
130                 if (DEF[blockIndex].count(*inlIter) == 0) {
131                     INL[blockIndex].insert(*inlIter);
132                 }
133                 change = true;
134             }
135         }
136     }
137 }
138 }
139
140
141 /*cout << "INL:" << std::endl;
142 printVectorOfSets(INL);
143 cout << "\nOUTL:" << std::endl;
144 printVectorOfSets(OUTL);
145 cout << "\nDEF:" << std::endl;
146 printVectorOfSets(DEF);
147 cout << "\nUSE:" << std::endl;
148 printVectorOfSets(USE);
149 cout << "*****";*/
150
151
152
153 funcOUTL[fbiter->first] = OUTL;
154 funcINL[fbiter->first] = INL;
155
156 //用来将 blocks 中的每个 Block 对象转换为 IBlock 对象,并将这些 IBlock 对象添加到 iBlocks
向量中
157 //都初始化为 非待用, 非活跃VarInfomation(-1, false)
158 for (vector<Block>::iterator iter = blocks.begin(); iter != blocks.end(); iter++) {
159     IBlock iBlock;
160     iBlock.next1 = iter->next1;
161     iBlock.next2 = iter->next2;

```

```

162     iBlock.name = iter->name;
163     for (vector<Quaternary>::iterator qIter = iter->codes.begin(); qIter != iter->codes.end
164           (); qIter++) {
165         iBlock.codes.push_back(QuaternaryWithInfo(*qIter, VarInfomation(-1, false),
166           VarInfomation(-1, false), VarInfomation(-1, false)));
167     }
168     iBlocks.push_back(iBlock);
169 }
170
171 vector<map<string, VarInfomation>> > symTables; // 每个基本块对应一张符号表
172 // 初始化符号表
173 // 对于每一个基本快，都找出它里面涉及到的变量 初始状态为 （非待用，非活跃）
174 /*Symbol Table for Block 0:
175 Variable: T5, Info : {-1, 0}
176 Variable: T6, Info : {-1, 0}
177 Variable: a, Info : {-1, 0}*/
178 for (vector<Block>::iterator biter = blocks.begin(); biter != blocks.end(); biter++) { // 遍
179   历每一个基本块
180     map<string, VarInfomation> symTable;
181     for (vector<Quaternary>::iterator citer = biter->codes.begin(); citer != biter->codes.
182           end(); citer++) { // 遍历基本块中的每个四元式
183         if (citer->op == "j" || citer->op == "call") {
184             // pass
185         }
186         else if (citer->op[0] == 'j') { // j>= j<=, j==, j!=, j>, j<
187             if (isVar(citer->src1)) {
188                 symTable[citer->src1] = VarInfomation{ -1, false };
189             }
190             if (isVar(citer->src2)) {
191                 symTable[citer->src2] = VarInfomation{ -1, false };
192             }
193         }
194         else {
195             if (isVar(citer->src1)) {
196                 symTable[citer->src1] = VarInfomation{ -1, false };
197             }
198             if (isVar(citer->src2)) {
199                 symTable[citer->src2] = VarInfomation{ -1, false };
200             }
201             if (isVar(citer->des)) {
202                 symTable[citer->des] = VarInfomation{ -1, false };
203             }
204         }
205     }
206     symTables.push_back(symTable);
207 }
208
209 cout << "*****symboltables*****" <<
210 endl;

```



```

208 // 循环遍历 symTables 中的每个符号表
209 for (size_t i = 0; i < symTables.size(); ++i) {
210     cout << "Symbol Table for Block " << i << ":" << endl;
211
212     // 获取当前基本块的符号表
213     const auto& symTable = symTables[i];
214
215     // 打印符号表中的每个变量及其信息
216     for (const auto& entry : symTable) {
217         cout << "Variable: " << entry.first << ", Info: {" << entry.second.next << ", " <<
entry.second.active << "}" << endl;
218     }
219
220     // 添加分隔符以区分不同的基本块
221     cout << "-----" << endl;
222 }
223 cout << "*****symboltables*****" <<
endl;
224
225
226 //更改出基活跃变量为 (-1, y)
227 int blockIndex = 0;
228 for (vector<set<string> >::iterator iter = OUTL.begin(); iter != OUTL.end(); iter++,
blockIndex++) { //遍历每个基本块的活跃变量表
229     for (set<string>::iterator viter = iter->begin(); viter != iter->end(); viter++) { //遍历
活跃变量表中的变量
230         symTables[blockIndex][*viter] = VarInfomation{ -1,true };
231     }
232 }
233
234
235
236
237 blockIndex = 0;
238 //计算每个四元式的待用信息和活跃信息
239 //symTables是基于变量的 会一步一步不加以备份地被修改
240 // citer是关于四元式的，数据结构为 四元式，右操作数，右操作数，左值
241
242 for (vector<IBlock>::iterator ibiter = iBlocks.begin(); ibiter != iBlocks.end(); ibiter++,
blockIndex++) { //遍历每一个基本块
243     int codeIndex = ibiter->codes.size() - 1;
244     //从后往前
245     //citer //(get,_,_,a)^(,^(,^(1,y) 这个形式 左操，右操，左值
246     for (vector<QuaternaryWithInfo>::reverse_iterator citer = ibiter->codes.rbegin(); citer
!= ibiter->codes.rend(); citer++, codeIndex--) { //逆序遍历基本块中的代码
247         if (citer->q.op == "j" || citer->q.op == "call") {
248             //pass
249         }
250         else if (citer->q.op[0] == 'j') { //j>= j<=,j==,j!=,j>,j<
251             if (isVar(citer->q.src1)) {
252                 citer->info1 = symTables[blockIndex][citer->q.src1]; //citer是基于式子的

```

```

253         symTables[blockIndex][citer->q.src1] = VarInfomation{ codeIndex, true };    //
symTables是基于变量，哪里用到，就改链
254     }
255     if (isVar(citer->q.src2)) {
256         citer->info2 = symTables[blockIndex][citer->q.src2];
257         symTables[blockIndex][citer->q.src2] = VarInfomation{ codeIndex, true };
258     }
259 }
260 else {
261     if (isVar(citer->q.src1)) {
262         citer->info1 = symTables[blockIndex][citer->q.src1];
263         symTables[blockIndex][citer->q.src1] = VarInfomation{ codeIndex, true };
264     }
265     if (isVar(citer->q.src2)) {
266         citer->info2 = symTables[blockIndex][citer->q.src2];
267         symTables[blockIndex][citer->q.src2] = VarInfomation{ codeIndex, true };
268     }
269     if (isVar(citer->q.des)) {
270         citer->info3 = symTables[blockIndex][citer->q.des];    // 先写进待用,活跃信息表中
271         symTables[blockIndex][citer->q.des] = VarInfomation{ -1, false };    // 左值写完以后,
就清空啦 变成 (-1, -1)
272     }
273 }
274 }
275 }
276
277 funcIBlocks[fbiter->first] = iBlocks;
278 outputIBlocks();
279 cout << "next*****" << endl;
280 }
281 cout << "*****iBlocks
*****" << endl;
282 }

```

3.3.3 目标代码生成的总控程序

1. 初始化和准备工作:

函数首先检查当前处理的函数 (nowFunc)，如果是主程序 ("program")，则可以执行特定初始化操作。它维护了两个变量 arg_num 和 par_num 来跟踪函数调用的实参个数和形参个数。par_list 用来收集函数调用时使用的实参列表，包括实参名和其活跃状态。

2. 处理输入变量 (IN 集):

函数遍历当前基本块的 IN 集 (in1)，这是在基本块开始前必须已经赋值并且

在基本块内部或之后某处使用的变量集合。对于 IN 集中的每一个变量，它使用一个集合 Avalue 来存储哪些寄存器当前存储了该变量的值。

3. 寄存器初始化:

清空空闲寄存器列表 freeReg 并初始化，假设有 8 个通用寄存器可用。

4. 生成代码的开头:

每个基本块开始处添加标签，如 nowIBlock->name + ":"。如果是函数的第一个基本块且函数是 main，则设置栈顶指针 top。否则，将返回地址 \$ra 存入栈中，以便在函数返回时能够正确恢复执行流。

5. 遍历基本块中的代码:

对于基本块内的每条四元式，使用 generateCodeForQuatenary 函数生成相应的汇编指令。如果当前指令是基本块的最后一条且是控制语句（如跳转、函数调用、返回等），则首先保存出口活跃变量（调用 storeOutLiveVar），然后生成控制语句的代码。如果最后一条指令不是控制语句，则先生成这条指令的代码，然后保存出口活跃变量。

6. 控制语句和非控制语句的处理:

控制语句需要特别处理，因为它们可能会改变执行流，所以在执行这些语句之前需要确保所有需要的数据都已正确保存。非控制语句如赋值操作则较为直接，首先执行指令，然后根据需要保存状态。

```
1 void ObjectCodeGenerator::generateCodeForBaseBlocks(int nowBaseBlockIndex) {
2     int arg_num = 0; // par的实参个数
3     int par_num = 0; // get的形参个数
4     list<pair<string, bool>> par_list; // 函数调用用到的实参集list<实参名,是否活跃>
5
6     if (nowFunc == "program") {
7         int a = 1;
8     }
9
10    Avalue.clear();
11    Rvalue.clear();
12    set<string>& inl = funcINL[nowFunc][nowBaseBlockIndex];
13    for (set<string>::iterator iter = inl.begin(); iter != inl.end(); iter++) {
14        Avalue[*iter].insert(*iter);
15    }
16
17    // 初始化空闲寄存器
18    // 8个寄存器
19    freeReg.clear();
20    for (int i = 0; i <= 7; i++) {
```

```

21     freeReg.push_back(string("$s") + to_string(i));
22 }
23
24 objectCodes.push_back(nowIBlock->name + ":");
25 if (nowBaseBlockIndex == 0) {
26     if (nowFunc == "main") {
27         top = 8;
28     }
29     else {
30         objectCodes.push_back("sw $ra 4($sp); // 把返回地址压栈 把$ra存到4($sp)");
31         top = 8;
32     }
33 }
34
35 for (vector<QuaternaryWithInfo>::iterator cIter = nowIBlock->codes.begin(); cIter !=
36     nowIBlock->codes.end(); cIter++) { // 对基本块内的每一条语句
37     nowQuaternary = *cIter;
38     // 如果是基本块的最后一条语句
39     if (cIter + 1 == nowIBlock->codes.end()) {
40         // 如果最后一条语句是控制语句，则先将出口活跃变量保存，再进行跳转(j, call, return)
41         if (isControlOp(cIter->q.op)) {
42             storeOutLiveVar(funcOUTL[nowFunc][nowBaseBlockIndex]);
43             generateCodeForQuaternary(nowBaseBlockIndex, arg_num, par_num, par_list);
44         }
45         // 如果最后一条语句不是控制语句（是赋值语句），则先计算，再将出口活跃变量保存
46         else {
47             generateCodeForQuaternary(nowBaseBlockIndex, arg_num, par_num, par_list);
48             storeOutLiveVar(funcOUTL[nowFunc][nowBaseBlockIndex]);
49         }
50     }
51     else {
52         generateCodeForQuaternary(nowBaseBlockIndex, arg_num, par_num, par_list);
53     }
54 }
55 }

```

3.3.4 目标代码生成函数

函数 `generateCodeForQuaternary` 负责将中间代码的四元式转换为目标代码（汇编语言），同时处理变量的寄存器分配和释放。以下是函数的详细解释，按照操作类型分类：

1. 错误检查

在转换任何四元式之前，函数首先检查四元式中使用的变量是否已被赋值。如

果一个变量在使用前没有赋值，函数会输出错误信息。这是为了确保程序的正确性和避免运行时错误。

2. 跳转处理

对于跳转指令（如无条件跳转 `j` 和条件跳转如 `j>`, `j<` 等），函数会转换为相应的汇编指令。无条件跳转直接转换为 `j` 指令。条件跳转会先对比较操作符进行解析，如 `j>=` 转换为汇编指令 `bge`，然后为操作数分配寄存器，并生成完整的跳转代码。

3. 函数调用和参数处理

对于 `par` 操作（表示函数参数），参数名和活跃状态被加入 `par_list`。在处理函数调用 (`call`) 操作时，函数将参数逐一压栈，并更新堆栈指针。随后，进行函数跳转 (`jal`) 并在函数执行完毕后恢复堆栈指针。

4. 返回指令

返回操作 (`return`) 根据返回值类型（数字或变量）设置相应的返回寄存器 (`$v0`)。对于主函数 `main`，执行跳转到程序结束；对于其他函数，从栈中恢复返回地址并通过 `jr` 指令跳回。

5. 赋值操作

对于赋值操作 (`=`)，函数检查赋值的源是否是特殊位置（如函数返回值），然后为源变量分配寄存器，并更新变量与寄存器之间的映射关系。

6. 算术运算

对于基本算术运算 (`+`, `-`, `*`, `/`)，函数为操作数分配寄存器，并生成相应的算术指令。对于除法，还需处理寄存器中的结果。

7. 寄存器管理

在处理每一个四元式后，函数检查操作数的活跃状态。如果一个变量不再活跃，它所占用的寄存器会被释放，以供其他变量使用。这是基于之前建立的待用/活跃信息表，确保寄存器的高效使用。

对于函数栈帧的处理是非常重要的环节，我将进行详细地解释：

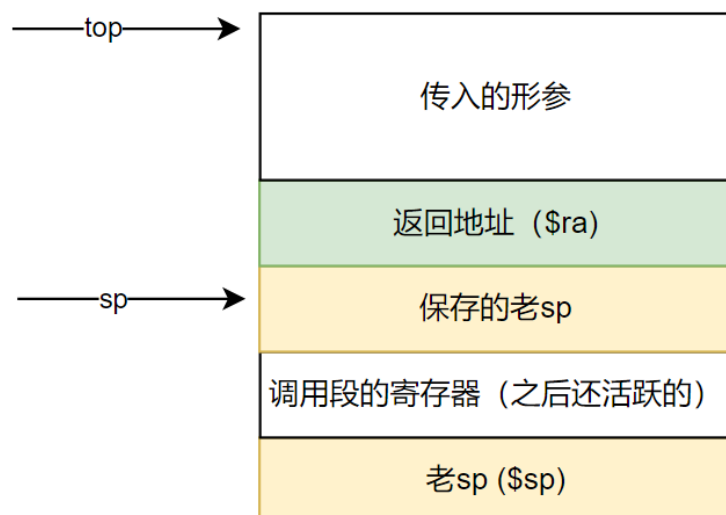


图 3: 函数栈帧

我以这个函数调用从 `main` 中调用 `demo`，再从 `demo` 中返回 `i` 为例，来讲解我设计的栈帧是如何工作的：

在进入一个函数之前，我们检测到下一句是控制语句 `call`，那我们首先检查一下这个基本块的所有变量，如果是出基活跃变量，我们就从 `Value` 寄存器中查找，将它保存在寄存器中。然后我们执行 `call` 语句，因为在 `call` 语句之前会有 `par` 语句，我们把要用到的形参放入了参数列表。那 `call` 语句的时候我们就控制 `sp` 的偏移量，将参数列表中的参数全部放入传入的形参这个区域。并保存老 `sp` 到这一单元，同时 `sp` 也更新跳转到这一单元。随后我们再加上跳转语句 `jal`，`jal` 在 `mips` 编译器中会自动将当前的 `pc` 存入到寄存器 `$ra` 中。然后我们加上一句恢复 `PC` 的代码，也就是从保存老 `sp` 的这个单元中取得数赋值到当前的 `sp` 中，完成栈帧的恢复。

`jal` 之后，我们就会进入到 `demo` 中，我们会判断现在程序刚进入一个函数基本块中，我们会从 `ra` 中得到返回地址，存到 `sw $ra 4($sp)` 当中。然后，我们会翻译 `get` 函数，就是将之前保存的形参按需取出。

在执行 `return` 语句，形如 `(add, $v0, $zero, $s4)` 时候，我们需要分情况讨论，如果返回的是纯数字，我们可以直接把数字放到 `$v0` 中；如果返回的是变量，我们就将变量所在的寄存器存入到 `$v0` 中。然后再加入 `lw $ra 4($sp)` 和 `jr $ra`。也就是说将 `4($sp)` 中的数据加载到 `$ra` 中，再进行跳转。这样 `PC` 也重新进行了更新，栈帧也进行了恢复。

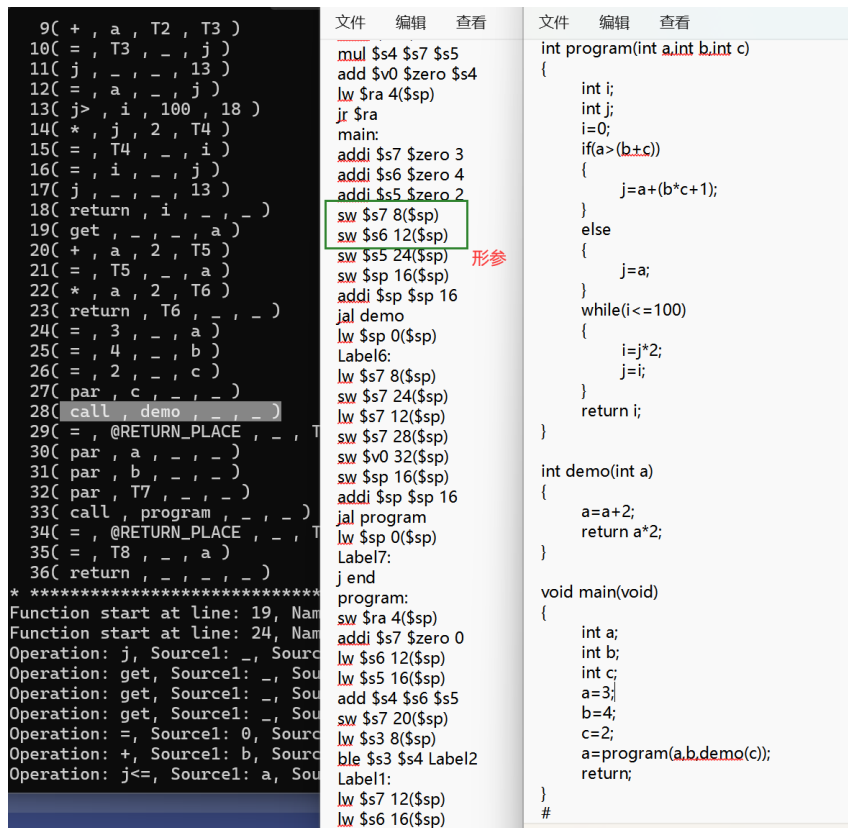


图 4: 形参调用示例讲解

```

1 void ObjectCodeGenerator::generateCodeForQuaternary (int nowBaseBlockIndex, int &arg_num, int &
  par_num, list<pair<string, bool>> &par_list) {
2   if (nowQuaternary->q.op[0] != 'j' && nowQuaternary->q.op != "call") {
3     if (isVar(nowQuaternary->q.src1) && Avalue[nowQuaternary->q.src1].empty()) { // AValue[A]={
4       R1,R2,A} 变量A存放的位置 ; RValue[R]={A,B} 寄存器R里存储着A和B
5       outputError(string("变量") + nowQuaternary->q.src1 + "在引用前未赋值");
6     }
7     if (isVar(nowQuaternary->q.src2) && Avalue[nowQuaternary->q.src2].empty()) {
8       outputError(string("变量") + nowQuaternary->q.src2 + "在引用前未赋值");
9     }
10  }
11
12  if (nowQuaternary->q.op == "j") { //j就直接沿用了, 因为之前已经标好了label
13    objectCodes.push_back(nowQuaternary->q.op + " " + nowQuaternary->q.des);
14  }
15  else if (nowQuaternary->q.op[0] == 'j') { //j>= j<=, j==, j!=, j>, j< 需要有比较再跳转 (j<=, a,
16    T0, Label12) 是的话就跳转到label12, 不是的话就继续往后顺序做。但你不要管这个逻辑, 你只需要分配寄存器
17    string op;
18    if (nowQuaternary->q.op == "j>=")
19      op = "bge";
20    else if (nowQuaternary->q.op == "j>")
21      op = "bgt";
22    else if (nowQuaternary->q.op == "j==")

```

```

22     op = "beq";
23 else if (nowQuaternary->q.op == "j!=")
24     op = "bne";
25 else if (nowQuaternary->q.op == "j<")
26     op = "blt";
27 else if (nowQuaternary->q.op == "j<=")
28     op = "ble";
29 string pos1 = allocateReg(nowQuaternary->q.src1); // 还是一样的给左右两个操作数分配寄存器,
    如果是数字就addi $s7 $zero 3 不是数字,能沿用的就沿用
30 string pos2 = allocateReg(nowQuaternary->q.src2);
31
32 objectCodes.push_back(op + " " + pos1 + " " + pos2 + " " + nowQuaternary->q.des); // bgt
    $s7 $s6 Label5
33 if (!nowQuaternary->info1.active) {
34     releaseVar(nowQuaternary->q.src1);
35 }
36 if (!nowQuaternary->info2.active) {
37     releaseVar(nowQuaternary->q.src2);
38 }
39 }
40 else if (nowQuaternary->q.op == "par") { // par 是参数传递的操作类型。在函数调用过程中, 参数
    需要传递给被调用函数, 而 par 操作就是为了处理这一过程。
41     par_list.push_back(pair<string, bool>(nowQuaternary->q.src1, nowQuaternary->info1.active));
42 }
43 // 处理函数调用的栈帧
44 // 将形参压栈
45 else if (nowQuaternary->q.op == "call") { //( call , demo , _ , _ )
46     /* 将参数压栈 */
47     for (list<pair<string, bool>>::iterator aiter = par_list.begin(); aiter != par_list.end()
        ; aiter++) {
48         string pos = allocateReg(aiter->first);
49         objectCodes.push_back(string("sw ") + pos + " " + to_string(top + 4 * (++arg_num + 1)) +
            "$sp"); // 把c压栈 先自增 sw $s5 24($sp)
50         if (!aiter->second) { // 看看这个形参后面还active不
51             releaseVar(aiter->first);
52         }
53     }
54     /* 更新$sp */
55     objectCodes.push_back(string("sw $sp ") + to_string(top) + "($sp)"); // sw $sp 16($sp)
    把$sp存到 16($sp)中 存老sp
56     objectCodes.push_back(string("addi $sp $sp ") + to_string(top)); // addi $sp $sp 16
57
58     /* 跳转到对应函数 */
59     objectCodes.push_back(string("jal ") + nowQuaternary->q.src1);
60
61     /* 恢复现场 */
62     objectCodes.push_back(string("lw $sp 0($sp)")); // 下载0($sp)中的数据到sp, 也就是说取得了
    老sp。也就是恢复了现场
63 }
64 else if (nowQuaternary->q.op == "return") {
65     if (isNum(nowQuaternary->q.src1)) { // 返回值为数字

```



```

66     objectCodes.push_back("addi $v0 $zero " + nowQuaternary->q.src1);
67 }
68 else if (isVar(nowQuaternary->q.src1)) { // 返回值为变量
69     set<string>::iterator piter = Avalue[nowQuaternary->q.src1].begin();
70     if ((*piter)[0] == '$') {
71         objectCodes.push_back(string("add $v0 $zero ") + *piter);
72     }
73     else {
74         objectCodes.push_back(string("lw $v0 ") + to_string(varOffset[*piter]) + "($sp)");
75     }
76 }
77 if (nowFunc == "main") {
78     objectCodes.push_back("j end");
79 }
80 else {
81     objectCodes.push_back("lw $ra 4($sp)");
82     objectCodes.push_back("jr $ra");
83 }
84 }
85 else if (nowQuaternary->q.op == "get") {
86     // varOffset[nowQuaternary->q.src1] = 4 * (par_num++ + 2);
87     varOffset[nowQuaternary->q.des] = top;
88     top += 4;
89     Avalue[nowQuaternary->q.des].insert(nowQuaternary->q.des);
90 }
91 else if (nowQuaternary->q.op == "=") { // (=,3,_,a) 等于号都是这样的
92     // Avalue[nowQuaternary->q.des] = set<string>();
93     string src1Pos;
94     if (nowQuaternary->q.src1 == "@RETURN_PLACE") { // ( = , @RETURN_PLACE , _ , T8 )
95         src1Pos = "$v0";
96     }
97     else { // ( = , T8 , _ , a )   (=,3,_,a)   addi $s7 $zero 3
98         src1Pos = allocateReg(nowQuaternary->q.src1); // dest直接沿用操作数的, 应为是赋值语句
99     }
100     Rvalue[src1Pos].insert(nowQuaternary->q.des);
101     Avalue[nowQuaternary->q.des].insert(src1Pos);
102 }
103 else { // + - * / // 有完整的三个   (+,b,c,T0)   T0=b+c   (+,a,2,T5)
104     string src1Pos = allocateReg(nowQuaternary->q.src1);
105     string src2Pos = allocateReg(nowQuaternary->q.src2);
106     string desPos = getReg();
107     if (nowQuaternary->q.op == "+") {
108         objectCodes.push_back(string("add ") + desPos + " " + src1Pos + " " + src2Pos);
109     }
110     else if (nowQuaternary->q.op == "-") {
111         objectCodes.push_back(string("sub ") + desPos + " " + src1Pos + " " + src2Pos);
112     }
113     else if (nowQuaternary->q.op == "*") {
114         objectCodes.push_back(string("mul ") + desPos + " " + src1Pos + " " + src2Pos);
115     }
116     else if (nowQuaternary->q.op == "/") {

```

```

117     objectCodes.push_back(string("div ") + src1Pos + " " + src2Pos);
118     objectCodes.push_back(string("mflo ") + desPos);
119 }
120 if (!nowQuaternary->info1.active) {    // 操作数1不再活跃，就是释放寄存器..这里就体现了待用活
    跃信息表的作用了
121     releaseVar(nowQuaternary->q.src1);
122 }
123 if (!nowQuaternary->info2.active) {    // 操作数2不再活跃，就是释放寄存器
124     releaseVar(nowQuaternary->q.src2);
125 }
126 }
127 }

```

3.3.5 寄存器分配算法

1. 为操作数分配寄存器

首先检查操作数是否为数字。如果是数字，则直接分配一个新寄存器，并生成一个立即数加载指令 (addi)。如果操作数是一个变量，则会检查这个变量是否已经存在于某个寄存器中。如果存在，直接使用该寄存器；如果不存在，这意味着变量可能是函数的形参，需要从堆栈加载到新分配的寄存器中。

2. 为目标变量分配寄存器

在类似语句 `op A, B,C` 即 `A=B+C` 中，我们会尽可能沿用 `B` 的寄存器，我们会查看存储 `B` 的寄存器中之后 `B` 一个变量，且 `B` 以后不活跃，我们就给把 `B` 的寄存器给 `A`。如果不成功，我们会尽可能用空闲的寄存器，如果当前已经没有空闲的寄存器。我们就抢占要在最远的将来才会引用到或不会引用到的寄存器，给 `A` 存。

3. 生成存数指令

当寄存器内容需要被更新或变量在内存中的值需要刷新时，编译器会生成存数指令。这通常发生在变量的值将要被覆盖前，或者当变量在程序的后续执行中仍然需要使用时。编译器会检查变量是否还保存在寄存器之外的其他位置。如果变量未来不会被引用，或者变量同时存储在其他位置，不需要生成存数指令。如果变量是基本块出口后活跃的，则必须确保其值在寄存器被释放前保存到内存中。

4 调试分析

界面展示了目标代码生成的分析过程，基础交互界面如下图所示，我首先将展示将正确的文法和正确的源程序输入后的结果展示：

4.1 正确结果演示

4.1.1 输入文法

```
P ::= N declare_list
declare_list ::= declare declare_list
declare_list ::= declare
declare ::= int ID M A function_declare
declare ::= int ID var_declare
declare ::= void ID M A function_declare
A ::=
var_declare ::= ;
function_declare ::= ( parameter ) sentence_block
parameter ::= parameter_list
parameter ::= void
parameter_list ::= param
parameter_list ::= param , parameter_list
param ::= int ID
sentence_block ::= { inner_declare sentence_list }
inner_declare ::=
inner_declare ::= inner_var_declare ; inner_declare
inner_var_declare ::= int ID
sentence_list ::= sentence M sentence_list
sentence_list ::= sentence
sentence ::= if_sentence
sentence ::= while_sentence
sentence ::= return_sentence
sentence ::= assign_sentence
assign_sentence ::= ID = expression ;
return_sentence ::= return ;
return_sentence ::= return expression ;
while_sentence ::= while M ( expression ) A sentence_block
if_sentence ::= if ( expression ) A sentence_block
if_sentence ::= if ( expression ) A sentence_block N else M A sentence_block
N ::=
M ::=
expression ::= add_expression
expression ::= add_expression > add_expression
expression ::= add_expression < add_expression
expression ::= add_expression == add_expression
expression ::= add_expression >= add_expression
expression ::= add_expression <= add_expression
```

图 5: 文法

4.1.2 源程序文件

```
1 int a;
2 int b;
3 int program(int a, int b, int c)
4 {
5     int i;
6     int j;
7     i=0;
8     if(a>(b+c))
9     {
10         j=a+(b*c+1);
11     }
12     else
```

```

13 {
14     j=a;
15 }
16 while(i <=100)
17 {
18     i=j*2;
19     j=i;
20 }
21 return i;
22 }
23
24 int demo(int a)
25 {
26     a=a+2;
27     return a*2;
28 }
29
30 void main(void)
31 {
32     int a;
33     int b;
34     int c;
35     a=3;
36     b=4;
37     c=2;
38     a=program(a,b,demo(c));
39     return;
40 }
41 #

```

4.1.3 生成语法树

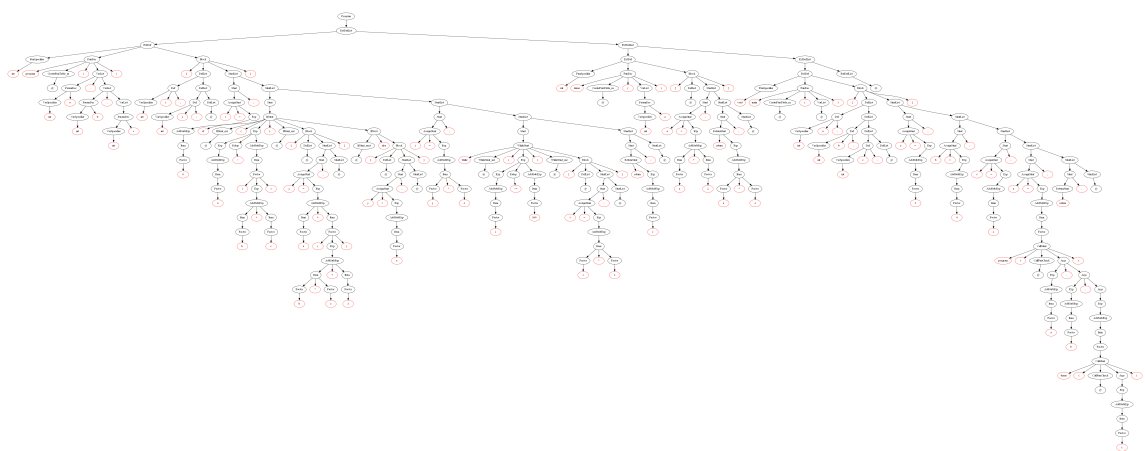


图 6: 生成的语法树

4.2 中间代码生成

```
0 (j,-,39)
1 (program,-,-)
2 (defpar,-,a)
3 (defpar,-,b)
4 (defpar,-,c)
5 (=,0,-,i)
6 (+,b,c,T0)
7 (j>,a,T0,10)
8 (=,0,-,T1)
9 (j,-,11)
10 (=,1,-,T1)
11 (j=,T1,0,18)
12 (j=-,13)|
13 (*,b,c,T2)
14 (+,T2,1,T3)
15 (+,a,T3,T4)
16 (=,T4,-,j)
17 (j,-,19)
18 (=,a,-,j)
19 (*,j,2,T5)
20 (=,T5,-,i)
21 (j<=,i,100,24)
22 (=,0,-,T6)
23 (j,-,25)
24 (=,1,-,T6)
25 (j=,T6,0,30)
26 (j,-,27)
27 (*,i,2,T7)
28 (=,T7,-,i)
29 (j,-,21)
30 (=,i,-,program_return_value)
31 (return,program_return_value,-,program)
32 (demo,-,-)
33 (defpar,-,a)
34 (+,a,2,T8)
35 (=,T8,-,a)
36 (*,a,2,T9)
37 (=,T9,-,demo_return_value)
38 (return,demo_return_value,-,demo)
39 (main,-,-)
40 (=,3,-,a)
41 (=,4,-,b)
42 (=,2,-,c)
43 (param,c,-)
44 (call,demo,-,T10)
45 (param,T10,-)
46 (param,b,-)
47 (param,a,-)
48 (call,program,-,T11)
49 (=,T11,-,a)
50 (return,-,-,main)
```

图 7: 中间代码生成

4.2.1 目标代码生成

将目标代码输入 MARS mips 编译器可以成功跑通，可喜可贺。

```

addi $sp,$sp,4096
addi $fp,$fp,4092
j LABEL_39
program :
subi $sp,$sp,4
sw $ra,0($sp)
subi $sp,$sp,4
sw $fp,0($sp)
move $fp,$sp
subi $sp,$sp,4
lw $t0,-4($fp)
addi $t0,$zero,0
subi $sp,$sp,4
lw $t1,-8($fp)
lw $t1,12($fp)
lw $t2,16($fp)
add $t1,$t1,$t2
lw $t3,8($fp)
bgt $t3,$t1,LABEL_10
subi $sp,$sp,4
lw $t4,-12($fp)
addi $t4,$zero,0
j LABEL_11
LABEL_10 :
addi $t4,$zero,1
LABEL_11 :
addi $t9,$zero,0
beq $t4,$t9,LABEL_18
subi $sp,$sp,4
lw $t5,-16($fp)
beq $t5,$t5,LABEL_13
LABEL_13 :
subi $sp,$sp,4
lw $t6,-20($fp)
lw $t6,12($fp)
mul $t6,$t6,$t2
subi $sp,$sp,4
lw $t7,-24($fp)
addi $t7,$t6,1
subi $sp,$sp,4
sw $t0,-4($fp)
lw $t0,-28($fp)
add $t0,$t3,$t7
subi $sp,$sp,4
sw $t1,-8($fp)
lw $t1,-32($fp)
move $t1,$t0
j LABEL_19
LABEL_18 :
move $t1,$t3
LABEL_19 :
subi $sp,$sp,4
sw $t4,-12($fp)
lw $t4,-36($fp)
addi $t8,$zero,2
mul $t4,$t1,$t8

```

图 8: 目标代码生成

4.3 错误处理展示

错误处理主要是在语法分析和语义分析中，属于上学期项目中处理过的内容，此处我仅举出几个例子：

- 示例 1：给程序下面的错误输入代码，我们明显可以看出 a 在未定义之前就被使用了：

```
        return a*2;
    }
    void main()
    {
        int b;
        int c;
        a=3;
        b=4;
        c=2;
        a=program(a,b,demo(c));
        return;
    }
```

图 9: 错误的源程序 1

程序在语义分析的时候会发现所悟，报错”a “未定义，属于参数未定义的错误。



```
Microsoft Visual Studio 调试  x  +  v
*****
          类C语言编译器
*****
          2051454 王逸菲
*****
系统默认读入类C语言文法文件grammar.txt

请输入要读入的文法文件名：（如果选择默认文件，请输入default）
default
已读入默认文件grammar.txt
正在对文法进行分析，请稍后...
已完成文法分析，可在../work_dir文件夹中查看
1. 产生式规则文件Grammar_Rules.txt
2. 初始闭包集文件Zero_Closure.txt
3. ACTION表和GOTO表Tables.csv.

系统默认读入源程序文件code_in.txt
请输入要进行词法、语法、语义分析的源程序文件名：（如果选择默认文件，请输入default）
error1.txt
正在读入源程序文件error1.txt
词法分析已结束，可在../work_dir文件夹中查看
1. 预处理代码文件Pre-Processed_Code.txt
2. 词法分析结果文件Lexical_Result.txt
3. 词法分析符号对应表Word_Label.txt

正在进行语法和语义分析，请稍后...
语义分析中发生错误："a"未定义
参数未定义
```

图 10: 未定义错误

- 示例 2：函数参数错误，定义和使用函数不一致。会有错误提示语义分析中发

生错误：函数调用参数过少，出错函数名为函数调用参数过多/过少

```
int program(int a,int b, int c)
{
    int i;
    int j;
    i=0;
    if(a>(b+c))
    {
        j=a+(b*c+1);
    }
    else
    {
        j=a;
    }
    i=j*2;
    while(i<=100)
    {
        i=i*2;
    }
    return i;
}

int demo(int a)
{
    a=a+2;
    return a*2;
}

void main()
{
    int a;
    int b;
    int c;
    a=3;
    b=4;
    c=2;
    a=program(a,b);
    return ;
}
```

图 11: 错误的源程序 2

```
*****
类C语言编译器
2051454 王逸菲
*****

系统默认读入类C语言文法文件grammar.txt

请输入要读入的文法文件名：（如果选择默认文件，请输入default）
default
已读入默认文件grammar.txt
正在对文法进行分析，请稍后...
已完成文法分析，可在../work_dir文件夹中查看
1. 产生式规则文件Grammar_Rules.txt
2. 初始闭包集文件Zero_Closure.txt
3. ACTION表和GOTO表Tables.csv.

系统默认读入源程序文件code_in.txt
请输入要进行词法、语法、语义分析的源程序文件名：（如果选择默认文件，请输入default）
error2.txt
正在读入源程序文件error2.txt
词法分析已结束，可在../work_dir文件夹中查看
1. 预处理代码文件Pre-Processed_Code.txt
2. 词法分析结果文件Lexical_Result.txt
3. 词法分析符号对应表Word_Label.txt

正在进行语法和语义分析，请稍后...
语义分析中发生错误：函数调用参数过少，出错函数名为    program
函数调用参数过多/过少
```

图 12: 函数调用参数错误

5 调试分析

5.1 时间复杂度分析

5.1.1 语义分析 parser 函数

parser 函数涉及到对词法符号的获取、语法规则的解析、状态转移、中间代码翻译、栈操作以及错误处理。因此考虑其时间复杂度比较繁琐。主要从词法符号的获取、状态转移和中间代码翻译的角度进行分析：

1、Token 获取：lexergetNextLexical(temp_token) 的时间复杂度与输入源代码的长度有关，为 $O(N)$ ，其中 N 是输入源代码的长度。

2、归约操作和翻译函数调用：归约操作和对应的翻译函数调用的复杂度取决于语法规则和翻译函数的具体实现。在最坏情况下，可能需要对每个输入 Token 进行一次归约操作和翻译函数调用。

因此综合考虑，该函数的时间复杂度可以粗略地表示为 $O(NM)$ ，其中 N 是输入源代码的长度， M 是语法规则的数量。

5.1.2 基本块划分

基本块划分基本块的划分是编译过程中的一项关键步骤，它主要依据函数的入口点开始。在此过程中，通过确定每个基本块的入口和出口来组织代码流。每个基本块由一系列顺序执行的四元式组成，直到遇到跳转或分支指令为止。基本块内的四元式数量直接影响基本块划分的时间复杂度。具体地，如果一个函数包含 n 个四元式，则基本块划分的时间复杂度为 $O(n)$ 。

5.1.3 目标代码生成

目标代码的生成过程涉及对已划分的函数块和基本块的遍历，针对各个带附加信息的四元式调用特定的代码生成函数来生成机器代码。这一阶段中，寄存器分配是一个经常执行的操作，其目的是为四元式中的变量找到合适的寄存器存储位置。

寄存器分配策略首先涉及遍历局部变量和全局变量区域，以识别所有变量的存储需求。之后，它会检查寄存器池以确定空闲寄存器。如果所有寄存器都已占

用，将采用最近最少使用（LRU）算法来选择一个寄存器进行释放和重用。在最坏情况下，如果局部变量区包含 m 个条目，全局变量区包含 k 个条目，寄存器区包含 r 个寄存器，则寄存器分配的时间复杂度为 $O(m+k+r)$ 。考虑到这三个区域的大小通常受到严格限制，实际的时间复杂度应该在可管理的范围内。

整体上，目标代码生成的过程是遍历每个四元式并对其进行处理。如果假设每个四元式处理时都涉及到寄存器分配，那么在有 n 个四元式的情况下，目标代码生成的整体时间复杂度将是 $O(n \times (m + k + r))$ ，其中每个四元式的处理都可能触发寄存器的分配和重新分配。

5.2 遇到的问题及解决办法

5.2.1 函数声明和调用问题

在设计如何对函数声明和调用来进行语义分析的时候，起初遇到了困难，很难用合适的方式来呈现每个函数块的起始和终止的地方，这是因为产生式没有合适的标记，在编译时难以捕捉。

经过一段时间的设计与分析，最后我们成功实施了对函数声明和调用的语义分析，以明确显示各函数块的起始标志和终止标志，并在函数调用时展示函数名称和参数。在进行函数声明方面，我们采用以下产生式：

为了解决函数声明过程中的符号表定义问题，我们在函数参数列表 `params` 定义之前插入了一个 `mktable` 符号。这个符号的翻译动作是新建一张符号表，并生成一条函数起始语句（例如“`PROCEDURE MAIN:`”）。由于在 `mktable` 处无法得知函数的名称（`id`），我们使用回填技术。这里生成一个空语句，在最后归约整个函数声明语句时，将函数名回填至中间代码。

对于函数调用的处理，函数调用的文法产生式如下：

```
1 factor -> id ( argus )
2 argus ->
3 argus -> argu_list
4 argu_list -> expr pre_expr
5 pre_expr ->
6 pre_expr -> , expr pre_expr
```

在函数调用处，首先将参数依次输出，然后生成 `call` 指令调用函数。我们采用参数传递的方法，在归约参数产生式时将参数信息逐层向左传递。最后，当归约

函数调用语句时，按序输出参数信息，生成相应的中间代码。

为了确保函数调用处可以找到函数对应的入口地址，我们在 `mktable` 建立符号表时为函数和其在符号表列表中的位置建立映射。这样，在函数调用处，我们可以根据函数名找到函数对应的符号表，并从中获取函数入口地址、函数返回值等信息。

5.2.2 符号表内容重复冗余问题

在 `parser` 部分和符号表、中间代码生成器的调用关系上遇到问题，具体为将数字和标识符的符号插入符号表的过程中遇到问题。将插入函数放置在了判断是否需要获取新的词法符号的 `if` 语句的外层，就是 `while` 循环中，因此导致符号表出现了多余的内容。如下：

```
1  if (!islast_Token) {
2      if (lexer.getNextLexical(temp_token) == -1)
3      {
4          return -1;
5      }
6
7  }
8  // 将num和标识符插入到符号表中
9  if (temp_token.tag == Tag::id)
10 {
11     temp_token.attrs.var_attrs.place = -1;
12 }
13 else if (temp_token.tag == Tag::num)
14 {
15     if (symbol_table.table_map_list.back().count(temp_token.value))
16         temp_token.attrs.var_attrs.place = symbol_table.table_map_list.back()[temp_token.value];
17     else
18         temp_token.attrs.var_attrs.place = symbol_table.addVar(temp_token.value, ArgType::t_int_var, true, temp_token.value, "int inst");
19     temp_token.attrs.var_attrs.item_i = temp_token.attrs.var_attrs.place;
20     temp_token.attrs.var_attrs.table_i = symbol_table.table_list.size() - 1;
21     temp_token.attrs.var_attrs.type = ArgType::t_int_var;
22 }
```

因此，在此处应该将插入符号表的操作放置在获取下一个词法符号之后，`if` 语句中，防止将词法符号重复加入符号表中。

5.2.3 存在部分符号无法识别问题

在测试过程中发现，在词法分析器中存在部分符号无法识别的情况，比如布尔产生式中的 && 和 || 符号。因此需要在词法分析器中，增加相关语句的识别和存储。在识别到 & 或者 | 时候，读取下一个符号判断 &&、||，然后在响应的词法表中进行检索，添加词法 Token 的 Tag 标签值，无法查找到的符号置为 error。

```
1 else if (current_process == '|' || current_process == '&')
2 {
3     else if (current_process == '&') { //分为>和>=
4         while ((ret = file.Getnext(false)) != EndOfFile) {
5             if (file.next_char == '&') {
6                 e.word = "&&";
7                 break;
8             }
9             else {
10                file.Retract();
11                e.word = "&";
12                break;
13            }
14        }
15    }
16    else if (current_process == '|') {
17        while ((ret = file.Getnext(false)) != EndOfFile) {
18            if (file.next_char == '|') {
19                e.word = "||";
20                break;
21            }
22            else {
23                file.Retract();
24                e.word = "|";
25                break;
26            }
27        }
28    }
29 }
```

5.2.4 出基活跃变量计算错误

活跃变量的基本算法解释如下：

初始化和循环条件：

- **change** 是一个标志，用来指示是否有变量的 IN 集或 OUT 集在迭代中有变化。如果有变化，则需要继续迭代直到没有更多变化，这意味着达到了一个固定

点。

- `while (change)` 循环会持续执行，直到一个完整的遍历中没有发现任何 IN 集或 OUT 集的变化。

迭代基本块：

- 循环遍历每个基本块，并对每个基本块的 IN 集和 OUT 集进行更新。`blockIndex` 用作当前基本块的索引。

更新 OUT 集和 IN 集：

- 对于每个基本块，根据其控制流后继 (`next1` 和 `next2`) 更新 OUT 集：
 - 如果 `next1` 不是 -1（表示存在有效的后继基本块），则遍历该后继基本块的 IN 集，将所有变量添加到当前基本块的 OUT 集中。
 - 类似地处理 `next2`。
- 在将变量从后继基本块的 IN 集添加到当前基本块的 OUT 集后，检查这些变量是否不在当前基本块的 DEF 集中：
 - 如果一个变量不在 DEF 集中，这意味着它在基本块中没有被重新定义，因此它需要从当前基本块的 IN 集中可访问，以确保它在使用前是活跃的。

检查和标记变化：

- 每当 OUT 集有新元素被添加时，检查是否需要更新 IN 集。
- 如果 IN 集被更新（即添加了新元素），将 `change` 标志设为 `true`，表示需要进行另一轮迭代。

遇到的一些错误：

然而我在一开始处理的时候没有注意到两个基本块之间参数未必是完全一致的，可能会有下图这种情况，也就是说：出基活跃变量未必只有我后继基本块的活跃变量，也就是 `use` 这个变量组决定，也可能是我后继块的后继块，也会被用到。所以，我们需要的首先用 `use` 来初始化入基活跃变量，然后看在我的前驱块的 `def` 中有没有被定义，没有被新定义就说明是之前的块中定义的，那我们就把他放入

前驱块的入基活跃变量中去。然后用一个 `change` 的布尔标志来记录，出基有没有变化，反复地扫描，直到没有变化，这样就能够从后往前传递。

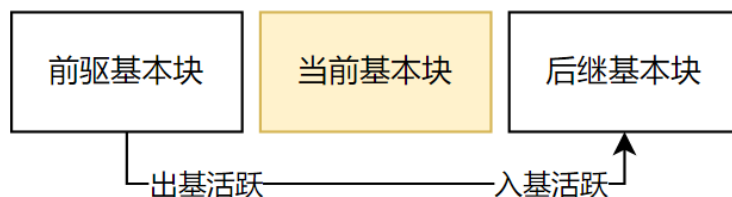


图 13: 基本块活跃变量关系

正确的代码如下：

```

1 bool change = true;
2 while (change) {
3     change = false;
4     int blockIndex = 0;
5     for (vector<Block>::iterator biter = blocks.begin(); biter != blocks.end(); biter++,
        blockIndex++) {
6         int next1 = biter->next1;
7         int next2 = biter->next2;
8         if (next1 != -1) { //有后续块
9             for (set<string>::iterator inlIter = INL[next1].begin(); inlIter != INL[next1].end();
                inlIter++) {
10                 if (OUTL[blockIndex].insert(*inlIter).second == true) {
11                     if (DEF[blockIndex].count(*inlIter) == 0) {
12                         INL[blockIndex].insert(*inlIter);
13                     }
14                     change = true;
15                 }
16             }
17         }
18         if (next2 != -1) { //有后续块
19             for (set<string>::iterator inlIter = INL[next2].begin(); inlIter != INL[next2].end();
                inlIter++) {
20                 if (OUTL[blockIndex].insert(*inlIter).second == true) {
21                     if (DEF[blockIndex].count(*inlIter) == 0) {
22                         INL[blockIndex].insert(*inlIter);
23                     }
24                     change = true;
25                 }
26             }
27         }
28     }
29 }

```

5.2.5 函数过程调用栈帧难点

我依旧使用这个例子来说明栈帧调用的难点：从 `main` 函数调用 `demo` 函数，并从 `demo` 中返回变量 `i` 的值。

首先，当程序即将进入一个新函数时，如果下一条指令是 `call`，我们首先检查该基本块的所有变量。对于那些在基本块结束时仍然活跃的变量，我们从 `Value` 寄存器中检索并保存这些变量的值到相应的寄存器中。之后，执行 `call` 指令前，通常会有 `par` 语句，我们将实参放入参数列表中。

执行 `call` 指令时，控制栈指针 `sp` 的偏移量，确保参数列表中的所有参数都被传递到目标函数的形参区域。同时，将当前的 `sp` 保存在一个新的栈帧单元中，并更新 `sp` 跳转到这个新单元。接着，通过 `jal` 指令，MIPS 编译器自动将当前的程序计数器 `pc` 存入 `$ra` 寄存器中，为函数调用做好准备。

当进入 `demo` 函数后，程序首先在新的基本块开始处执行，我们从 `$ra` 寄存器中获取返回地址，并将其存储到栈空间 `$ra 4($sp)` 中。随后，函数中需要的形参将从之前保存的状态中恢复。

当 `demo` 函数执行到 `return` 语句时，例如形如 `(add $v0, $zero, $s4)`，我们需要根据返回值的类型处理：如果是直接返回数字，我们直接将该数字赋值到 `$v0` 寄存器中；如果返回的是变量，则将该变量所在的寄存器的值赋给 `$v0`。随后执行 `lw $ra 4($sp)` 和 `jr $ra`，这两条指令负责从栈中恢复 `$ra` 寄存器的值并通过 `$ra` 跳转回调用函数，同时更新 `pc` 和恢复栈帧到调用前的状态。这样，栈帧的整个进入和退出过程就完整地展示了我的栈帧设计的工作原理。

在一开始写的时候，没有意识到栈帧和 `PC` 的区别，没有记录返回地址，我以为记录了老的 `sp` 就可以实现 `call` 和 `return`，但其实我们需要用 `ra` 寄存器来记录当前语句的 `PC` 值，同时也记录在老 `sp` 的上面一个存储单元当中。这一点我之前没有意识到，所以花了很多时间。通过这次的 `debug` 过程，我弥补了这一方面的概念混淆，非常有帮助。

6 用户使用说明

`cmd` 界面展示清晰友善，包括合理适时的提示输入：

1. 读取语法规则文件 `grammar.txt`

用户需要指定一个语法规则文件，该文件定义了编译器需要处理的语言的语法。默认情况下，如果未指定文件，则使用默认设置。

2. 处理语法规则文件

文件处理后生成三个主要文件：`Grammar_Rules.txt`, `Zero_Closure.txt`, `ACTION` 和 `GOTO` 表。

3. 读取源代码文件 `code_in.txt`

用户指定待编译的源代码文件。默认情况下，如未指定则使用默认配置。

4. 预处理源代码文件

预处理生成的文件位于 `.work_dir` 文件夹，包括：

- `Pre-Processed_Code.txt` —— 预处理后的代码。
- `Lexical_Result.txt` —— 词法分析结果。
- `Word_Label.txt` —— 包含词法标签和属性。

5. 进一步分析和处理

用户可以选择是否进一步处理源代码文件。如果继续，将生成以下文件：`Quaternion_List.txt` —— 中间代码四元数列表。

6. 生成语法分析树

生成的语法分析树会被保存为 `Parse_Tree.png`。

7. 生成目标代码

最终阶段是生成目标代码，保存为 `Target_Code.asm`。这一步骤将源代码转换为机器可执行的指令。

这个界面为用户提供了从语法分析到目标代码生成的完整编译流程。用户可以通过简单的命令行交互指定文件、控制处理步骤并生成所需的输出。


```

*****
类C语言编译器

2051454 王逸菲
*****

系统默认读入类C语言文法文件grammar.txt

请输入要读入的文法文件名：（如果选择默认文件，请输入default）
default
已读入默认文件grammar.txt
正在对文法进行分析，请稍后...
已完成文法分析，可在../work_dir文件夹中查看
1. 产生式规则文件Grammar_Rules.txt
2. 初始闭包集文件Zero_Closure.txt
3. ACTION表和GOTO表Tables.csv。

系统默认读入源程序文件code_in.txt
请输入要进行词法、语法、语义分析的源程序文件名：（如果选择默认文件，请输入default）
default
正在读入默认源程序文件code_in.txt
词法分析已结束，可在../work_dir文件夹中查看
1. 预处理代码文件Pre-Processed_Code.txt
2. 词法分析结果文件Lexical_Result.txt
3. 词法分析符号对应表Word_Lable.txt

正在进行语法和语义分析，请稍后...
语法和语义分析已结束，可在../work_dir文件夹中查看
1. 文法分析过程文件Analysis_Process.txt
2. 语义分析四元式序列文件Quaternion_List.txt。
是否需要生成归约语法树图片（y/n）
y
语法分析树已生成，可在../work_dir文件夹中查看
1. 语法树图片Parse_Tree.png。

正在进行目标代码生成，请稍后...
目标代码生成已结束，可在../work_dir文件夹中查看
1. 目标汇编代码文件Target_Code.asm。

程序结束！

```

图 14: cmd 界面展示

7 总结与收获

7.1 收获

在之前的学习过程中，我对中间代码生成的步骤和原理只有在课堂上被动地接触，了解了设计生成符号表，语法制导分析过程等方法。然而，对这些方法背后的深层原理，并没有深刻的理解，仅能在完成作业时按部就班地套用这些方法。通过本次个人大作业的完成，我通过亲自编写代码，对语法分析的每个环节有了更加深入的理解。

在目标代码生成的过程中，理解待用-活跃信息表的重要性对于有效地进行编译优化至关重要。在我之前的理论学习和做题经验中，我并未充分理解为什么需要构建这样的信息表。通常在手动解题时，遇到的编译情形相对简单，看似只需

简单地向后扫描代码即可直观判断哪些变量仍然活跃，以此决定哪些寄存器的内容可以继续使用，哪些可以被重新分配。

然而，随着我在编译原理课程中深入实践，我逐渐认识到在复杂的编译环境中，手动判断的方法远远不够精确和高效。真实世界的程序往往涉及大量变量和复杂的控制流，这使得仅仅通过简单扫描来确定变量是否活跃变得不可行。待用-活跃信息表的建立正是为了系统地解决这一问题。

待用-活跃信息表详细记录了每个变量在代码中的使用情况和活跃状态。通过这种方式，编译器能够精确地知道在任何给定时刻哪些变量是活跃的，哪些已经不再被需要。这一信息对于寄存器分配策略至关重要，因为有效的寄存器利用可以极大地提高程序的运行效率。例如，如果一个变量在某段代码后不再被访问，它占用的寄存器就可以安全地分配给其他变量，从而避免不必要的内存访问和寄存器溢出。

7.2 遇到的问题与解决思路

在目标代码生成器的具体代码编写过程中，我深刻考虑了实际的内存开销问题。特别是在处理大量临时变量时，这一点尤为重要。我通过优化存储结构和改进变量使用策略，有效减少了内存占用。

除此之外，我遇到了代码生成逻辑复杂性高的问题，特别是在实现跳转和循环结构时。通过反复测试和调试，我逐渐理解了如何有效地生成逻辑紧凑的目标代码。我学会了如何更好地控制程序流，使生成的代码不仅正确，而且效率高。

同时，在出基活跃变量的计算中，我参考了 [github](#) 上的部分学长代码，仿照着他的思路，利用 `change` 这个 `flag`，来控制是否更新结束。具体来说在进行编译优化中的活跃变量分析时，对活跃变量的初始识别并不仅限于直接后继基本块的使用（即 `use` 集合）。实际上，一个变量的活跃性可能不只是由当前块的直接后继决定，还可能受到更远后继块的影响。因此，在分析过程中，首先使用 `use` 集合来初始化每个基本块的入基活跃变量集合。接下来，我们需要检查该变量在前驱基本块中是否有定义（即检查 `def` 集合）。

如果一个变量在前驱基本块的 `def` 集合中没有新的定义，这表明它是在更早的基本块中定义的。因此，这个变量应该被加入到前驱块的入基活跃变量集合中。为了确保所有相关变量的活跃性得到正确处理，我们使用一个布尔标志 `change` 来追踪活跃变量集合是否发生变化。这个过程需要反复扫描，从后往前逐步传递和

更新活跃变量信息，直至整个流程中 `change` 标志不再变化，即没有新的活跃变量需要添加到集合中。这种方法确保了从后续基本块到当前基本块的活跃变量传递能够被准确捕捉和处理。

7.3 在实验过程中对课程的认识

总的来说，在本次编译原理课程的项目中，通过实现一个类 C 语言的编译器，我对编译原理这一学科的理解有了显著的深入。编译原理本质上是一门探讨如何设计并实现能被机器理解同时对程序员友好的编程语言的学科。通过亲手构建编译器，我不仅对 C 语言的语法有了更深刻的掌握，还深入了解了日常使用中编译器的内部工作原理。

从编译器的工作流程开始，包括词法分析、语法分析、语义分析及最终的目标代码生成，每个阶段都是将高级语言逐步转换为机器可以执行的语言的重要步骤。词法分析帮助我识别和分类程序中的各种基本元素，如标识符、关键字等。在语法分析阶段，我学会了如何根据语言的语法规则构建抽象语法树，这是理解程序结构的关键。接着，在语义分析中，我确保了语法结构的逻辑一致性，并处理了数据类型和表达式的计算。最后，在目标代码生成阶段，我将高级代码转化为机器语言，这一过程中涉及到的优化技术尤为关键，它们确保生成的代码既高效又紧凑。

通过这一实践过程，我不仅增强了对于编译过程各阶段的理论知识的理解，而且实际操作使我能够更好地加深了我对理论的掌握，对我之后计算机学科相关知识的学习有着积极的影响。